PRACTICAL 6

6. Develop an event management App. The App should suggest venues as per event and size of crowd.

1. server.js

```
const express = require("express");
const path = require("path");
const venues = require("./venues");

const app = express();
const PORT = process.env.PORT || 3000;
const eventTypes = ["Wedding", "Birthday", "Corporate", "Conference", "Party"];

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/api/venues", (req, res) => {
  res.json({ success: true, count: venues.length, venues });
});

app.post("/api/suggest", (req, res) => {
  const { eventType, crowdSize } = req.body || {};
  const size = Number(crowdSize);

  if (!eventTypes.includes(eventType)) {
    return res.status(400).json({
      success: false,
      message: "Please select a valid event type."
    });
  }

  if (!Number.isFinite(size) || size <= 0) {
    return res.status(400).json({
      success: false,
      message: "Crowd size must be a positive number."
    });
  }

  const matches = venues
    .filter((venue) =>
      venue.type.includes(eventType) &&
      size >= venue.minCapacity &&
      size <= venue.maxCapacity
    )
    .sort((a, b) => {
      const unusedCapacityA = a.maxCapacity - size;
      const unusedCapacityB = b.maxCapacity - size;
      return unusedCapacityA - unusedCapacityB || b.rating - a.rating;
    });

  const response = {
    success: true,
    count: matches.length,
    venues: matches
  };

  if (matches.length === 0) {
    response.message = "No exact venue found for this event and crowd size.";
  }

  res.json(response);
});

app.use((error, req, res, next) => {
  const isBadJson = error instanceof SyntaxError && error.status === 400;
  const status = isBadJson ? 400 : 500;
  const message = isBadJson ? "Please send valid JSON." : "Internal server error.";
  res.status(status).json({ success: false, message });
});

app.listen(PORT, () => {
  console.log(`EVENTORA is running at http://localhost:${PORT}`);
});
```

2. venues.js

```
const venues = [
  {
    id: 1,
    name: "Regal Crown Banquet",
    type: ["Wedding", "Party"],
    minCapacity: 120,
    maxCapacity: 500,
    location: "City Center",
    priceRange: "Premium",
    rating: 4.8,
    description: "An elegant banquet hall designed for grand celebrations."
  },
  {
    id: 2,
    name: "Emerald Grand Lawn",
    type: ["Wedding", "Birthday", "Party"],
    minCapacity: 250,
    maxCapacity: 900,
    location: "Riverside Avenue",
    priceRange: "Premium",
    rating: 4.7,
    description: "A spacious green lawn for memorable open-air occasions."
  },
  {
    id: 3,
    name: "Skyline Terrace",
    type: ["Birthday", "Corporate", "Party"],
    minCapacity: 40,
    maxCapacity: 150,
    location: "Central District",
    priceRange: "High",
    rating: 4.6,
    description: "A stylish rooftop setting with an impressive city view."
  },
  {
    id: 4,
    name: "Summit Conference Hall",
    type: ["Corporate", "Conference"],
    minCapacity: 70,
    maxCapacity: 300,
    location: "Knowledge Park",
    priceRange: "Moderate",
    rating: 4.7,
    description: "A professional hall with a focused and refined atmosphere."
  },
  {
    id: 5,
    name: "The Executive Forum",
    type: ["Corporate", "Conference"],
    minCapacity: 20,
    maxCapacity: 100,
    location: "Business Square",
    priceRange: "Moderate",
    rating: 4.5,
    description: "A smart business venue suited to meetings and seminars."
  },
  {
    id: 6,
    name: "Velvet Party Lounge",
    type: ["Birthday", "Party"],
    minCapacity: 30,
    maxCapacity: 120,
    location: "West End",
    priceRange: "High",
    rating: 4.6,
    description: "An intimate lounge with a lively yet sophisticated mood."
  },
  {
    id: 7,
    name: "Harmony Community Hall",
    type: ["Wedding", "Birthday", "Conference"],
    minCapacity: 80,
    maxCapacity: 350,
    location: "Garden Colony",
```

```
    priceRange: "Affordable",
    rating: 4.3,
    description: "A welcoming, versatile hall for value-conscious events."
  },
  {
    id: 8,
    name: "Pearl Celebration Studio",
    type: ["Birthday", "Corporate", "Party"],
    minCapacity: 10,
    maxCapacity: 60,
    location: "Lake Road",
    priceRange: "Affordable",
    rating: 4.4,
    description: "A charming small hall for close-knit celebrations."
  }
];

module.exports = venues;
```

3. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="EVENTORA suggests the right venue for your event and guest count.">
  <title>EVENTORA | Find Your Perfect Venue</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <a class="brand" href="#home" aria-label="EVENTORA home">
      <span class="brand-mark">E</span>
      <span>EVENTORA</span>
    </a>
    <nav aria-label="Main navigation">
      <a href="#home">Home</a>
      <a href="#finder">Find Venue</a>
    </nav>
  </header>

  <main>
    <section class="hero" id="home">
      <div class="hero-content">
        <p class="eyebrow">CURATED SPACES · MEMORABLE MOMENTS</p>
        <h1>Your Event Deserves<br>the <em>Perfect Place.</em></h1>
        <p class="hero-copy">
          Tell us your occasion and guest count. We'll suggest venues that fit.
        </p>
        <a class="button hero-button" href="#finder">Find a Venue <span>→</span></a>
      </div>
      <aside class="hero-note" aria-label="EVENTORA promise">
        <span>01</span>
        <p>Thoughtfully matched<br>for your occasion</p>
      </aside>
    </section>

    <section class="finder-section" id="finder">
      <div class="section-heading">
        <p class="eyebrow">VENUE CONCIERGE</p>
        <h2>Find Your Perfect Venue</h2>
        <p>Two simple details. A selection made for you.</p>
      </div>

      <form id="venue-form" class="finder-card" novalidate>
        <div class="form-group">
          <label for="event-type">Event Type</label>
          <div class="select-wrap">
            <select id="event-type" required>
              <option value="">Select an occasion</option>
              <option>Wedding</option>
              <option>Birthday</option>
              <option>Corporate</option>
              <option>Conference</option>
              <option>Party</option>
```

```
          </select>
        </div>
      </div>

      <div class="form-group">
        <label for="crowd-size">Crowd Size</label>
        <input id="crowd-size" type="number" min="1" placeholder="Number of guests" required>
      </div>

      <button class="button submit-button" type="submit">
        <span class="button-text">Suggest Venues</span>
        <span aria-hidden="true">→</span>
      </button>
      <p id="form-error" class="form-error" role="alert"></p>
    </form>

    <section class="results" aria-live="polite" aria-atomic="false">
      <div id="results-heading" class="results-heading">
        <p class="eyebrow">YOUR SELECTION</p>
        <h2>Recommended Venues</h2>
      </div>
      <div id="results-grid" class="results-grid">
        <div class="empty-state">
          <span class="empty-symbol">◇</span>
          <h3>Your perfect venue is waiting.</h3>
          <p>Choose your event details above to begin.</p>
        </div>
      </div>
    </section>
  </section>
 </main>

 <footer>
  <a class="brand footer-brand" href="#home"><span class="brand-mark">E</span> EVENTORA</a>
  <p>Find the perfect venue for every occasion.</p>
 </footer>

 <script src="app.js"></script>
</body>
</html>
```

4. public/style.css

```
:root {
  --navy: #09131f;
  --navy-light: #101f2e;
  --panel: #132536;
  --gold: #c9a45c;
  --gold-light: #e4c987;
  --cream: #f5f0e6;
  --muted: #a9b0b5;
  --border: rgba(201, 164, 92, 0.28);
  --shadow: 0 22px 55px rgba(0, 0, 0, 0.25);
}

* { box-sizing: border-box; }
html { scroll-behavior: smooth; }

body {
  margin: 0;
  color: var(--cream);
  background: var(--navy);
  font-family: Arial, Helvetica, sans-serif;
  line-height: 1.6;
}

a { color: inherit; text-decoration: none; }
h1, h2, h3, p { margin-top: 0; }
h1, h2, h3 {
  font-family: Georgia, "Times New Roman", serif;
  font-weight: 400;
}

.site-header {
  position: absolute;
  z-index: 5;
```

```
  top: 0;
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 28px max(6vw, 24px);
  border-bottom: 1px solid rgba(255, 255, 255, 0.08);
}

.brand {
  display: flex;
  align-items: center;
  gap: 12px;
  font-family: Georgia, "Times New Roman", serif;
  font-size: 1.1rem;
  letter-spacing: 0.18em;
}

.brand-mark {
  display: grid;
  width: 34px;
  height: 34px;
  place-items: center;
  color: var(--navy);
  background: var(--gold);
  border-radius: 50%;
  font-style: italic;
  letter-spacing: 0;
}

nav { display: flex; gap: 34px; }
nav a {
  color: #d8d9d6;
  font-size: 0.82rem;
  letter-spacing: 0.09em;
  text-transform: uppercase;
}
nav a:hover { color: var(--gold-light); }

.hero {
  position: relative;
  min-height: 720px;
  display: flex;
  align-items: center;
  padding: 150px max(8vw, 28px) 90px;
  overflow: hidden;
  background:
    linear-gradient(90deg, rgba(9, 19, 31, 0.98) 8%, rgba(9, 19, 31, 0.82) 58%, rgba(9, 19, 31, 0.92)),
    radial-gradient(circle at 82% 45%, #395267 0, #172b3d 25%, var(--navy) 55%);
}

.hero::after {
  position: absolute;
  right: -120px;
  width: 520px;
  height: 520px;
  content: "";
  border: 1px solid var(--border);
  border-radius: 50%;
  box-shadow: 0 0 0 70px rgba(201, 164, 92, 0.025),
              0 0 0 140px rgba(201, 164, 92, 0.02);
}

.hero-content { position: relative; z-index: 1; max-width: 790px; }
.eyebrow {
  margin: 0 0 18px;
  color: var(--gold-light);
  font-size: 0.72rem;
  font-weight: 700;
  letter-spacing: 0.22em;
}

h1 {
  margin-bottom: 26px;
  font-size: clamp(3.1rem, 7vw, 6rem);
```

```
  line-height: 1.03;
  letter-spacing: -0.035em;
}
h1 em { color: var(--gold-light); font-weight: 400; }
.hero-copy {
  max-width: 560px;
  margin-bottom: 36px;
  color: #c9ced1;
  font-size: 1.08rem;
}

.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 38px;
  min-height: 52px;
  padding: 0 27px;
  color: #111820;
  background: var(--gold);
  border: 1px solid var(--gold);
  border-radius: 2px;
  font-size: 0.78rem;
  font-weight: 700;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  cursor: pointer;
  transition: 180ms ease;
}
.button:hover { background: var(--gold-light); transform: translateY(-2px); }

.hero-note {
  position: absolute;
  z-index: 1;
  right: 7vw;
  bottom: 65px;
  display: flex;
  align-items: center;
  gap: 18px;
  color: #b8bdc0;
  font-size: 0.78rem;
  letter-spacing: 0.08em;
}
.hero-note span { color: var(--gold); font: 2.3rem Georgia, serif; }
.hero-note p { margin: 0; padding-left: 18px; border-left: 1px solid var(--border); }

.finder-section {
  padding: 95px max(7vw, 24px) 110px;
  background: #0d1a27;
}
.section-heading, .results-heading {
  max-width: 700px;
  margin: 0 auto 36px;
  text-align: center;
}
.section-heading h2, .results-heading h2 {
  margin-bottom: 10px;
  font-size: clamp(2.2rem, 5vw, 3.5rem);
}
.section-heading > p:last-child { color: var(--muted); }

.finder-card {
  position: relative;
  display: grid;
  grid-template-columns: 1fr 1fr auto;
  gap: 22px;
  max-width: 1040px;
  margin: 0 auto;
  padding: 34px;
  background: var(--panel);
  border: 1px solid var(--border);
  box-shadow: var(--shadow);
}

.form-group label {
```

```
  display: block;
  margin-bottom: 9px;
  color: #d5d7d6;
  font-size: 0.74rem;
  font-weight: 700;
  letter-spacing: 0.1em;
  text-transform: uppercase;
}

input, select {
  width: 100%;
  height: 54px;
  padding: 0 16px;
  color: var(--cream);
  background: #0d1b28;
  border: 1px solid rgba(255, 255, 255, 0.14);
  border-radius: 2px;
  outline: none;
  font: inherit;
}
select option { color: var(--cream); background: var(--navy); }
input:focus, select:focus {
  border-color: var(--gold);
  box-shadow: 0 0 0 3px rgba(201, 164, 92, 0.1);
}
.submit-button { align-self: end; min-width: 205px; }
.submit-button:disabled { cursor: wait; opacity: 0.72; transform: none; }
.form-error {
  position: absolute;
  top: calc(100% + 8px);
  left: 34px;
  color: #f0a9a9;
  font-size: 0.86rem;
}

.results { max-width: 1160px; margin: 110px auto 0; }
.results-heading { display: none; }
.results-heading.visible { display: block; }
.results-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 22px;
}
.venue-card, .empty-state {
  padding: 30px;
  background: var(--navy-light);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
.venue-card { transition: 180ms ease; }
.venue-card:hover { border-color: var(--gold); transform: translateY(-4px); }
.venue-topline, .venue-detail {
  display: flex;
  justify-content: space-between;
  gap: 14px;
}
.venue-location, .venue-rating {
  color: var(--gold-light);
  font-size: 0.76rem;
  letter-spacing: 0.08em;
  text-transform: uppercase;
}
.venue-card h3 { margin: 20px 0 9px; font-size: 1.55rem; line-height: 1.25; }
.venue-description { min-height: 52px; color: var(--muted); font-size: 0.9rem; }
.venue-types { color: #d9cdaf; font-size: 0.8rem; }
.venue-detail {
  margin-top: 21px;
  padding-top: 18px;
  border-top: 1px solid rgba(255, 255, 255, 0.09);
  font-size: 0.82rem;
}

.empty-state {
  grid-column: 1 / -1;
  padding: 58px 24px;
  text-align: center;
```

```
  border-style: dashed;
}
.empty-symbol { color: var(--gold); font-size: 2.5rem; }
.empty-state h3 { margin: 12px 0 6px; font-size: 1.7rem; }
.empty-state p { margin-bottom: 0; color: var(--muted); }

footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 30px max(6vw, 24px);
  color: var(--muted);
  border-top: 1px solid rgba(255, 255, 255, 0.08);
  font-size: 0.82rem;
}
footer p { margin: 0; }
.footer-brand { font-size: 0.9rem; }

@media (max-width: 850px) {
  .hero { min-height: 680px; }
  .hero-note { display: none; }
  .finder-card { grid-template-columns: 1fr 1fr; }
  .submit-button { grid-column: 1 / -1; }
  .results-grid { grid-template-columns: repeat(2, 1fr); }
}

@media (max-width: 560px) {
  .site-header { padding: 20px; }
  nav { gap: 16px; }
  nav a { font-size: 0.7rem; }
  .brand { gap: 8px; font-size: 0.85rem; }
  .brand-mark { width: 29px; height: 29px; }
  .hero { min-height: 650px; padding: 130px 22px 70px; }
  h1 { font-size: 3rem; }
  .finder-section { padding: 72px 18px 85px; }
  .finder-card, .results-grid { grid-template-columns: 1fr; }
  .finder-card { padding: 24px; }
  .submit-button { grid-column: auto; }
  .form-error { left: 24px; }
  .results { margin-top: 95px; }
  footer { flex-direction: column; gap: 18px; text-align: center; }
}
```

5. public/app.js

```
const form = document.querySelector("#venue-form");
const eventTypeInput = document.querySelector("#event-type");
const crowdSizeInput = document.querySelector("#crowd-size");
const errorText = document.querySelector("#form-error");
const resultsGrid = document.querySelector("#results-grid");
const resultsHeading = document.querySelector("#results-heading");
const submitButton = form.querySelector("button");
const buttonText = submitButton.querySelector(".button-text");

function showMessage(title, message) {
  resultsGrid.textContent = "";
  const box = document.createElement("div");
  box.className = "empty-state";

  const symbol = document.createElement("span");
  symbol.className = "empty-symbol";
  symbol.textContent = "◇";

  const heading = document.createElement("h3");
  heading.textContent = title;

  const text = document.createElement("p");
  text.textContent = message;

  box.append(symbol, heading, text);
  resultsGrid.appendChild(box);
}

function addText(parent, className, text, tag = "p") {
  const element = document.createElement(tag);
  element.className = className;
```

```
  element.textContent = text;
  parent.appendChild(element);
}

function displayVenues(venues) {
  resultsGrid.textContent = "";
  resultsHeading.classList.add("visible");

  venues.forEach((venue) => {
    const card = document.createElement("article");
    card.className = "venue-card";

    const topline = document.createElement("div");
    topline.className = "venue-topline";
    addText(topline, "venue-location", venue.location);
    addText(topline, "venue-rating", `★ ${venue.rating}`);

    addText(card, "venue-name", venue.name, "h3");
    addText(card, "venue-description", venue.description);
    addText(card, "venue-types", venue.type.join(" · "));

    const details = document.createElement("div");
    details.className = "venue-detail";
    addText(details, "", `${venue.minCapacity}–${venue.maxCapacity} guests`, "span");
    addText(details, "", venue.priceRange, "span");

    card.prepend(topline);
    card.appendChild(details);
    resultsGrid.appendChild(card);
  });
}

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  errorText.textContent = "";

  const eventType = eventTypeInput.value;
  const crowdSize = Number(crowdSizeInput.value);

  if (!eventType) {
    errorText.textContent = "Please select an event type.";
    return;
  }

  if (!Number.isFinite(crowdSize) || crowdSize <= 0) {
    errorText.textContent = "Please enter a valid positive crowd size.";
    return;
  }

  submitButton.disabled = true;
  buttonText.textContent = "Finding Venues...";
  showMessage("Searching our collection...", "Matching your occasion and guest count.");

  try {
    const response = await fetch("/api/suggest", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ eventType, crowdSize })
    });
    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.message || "Request failed");
    }

    if (data.count === 0) {
      resultsHeading.classList.remove("visible");
      showMessage("No exact venue found.", "Try changing the event type or guest count.");
    } else {
      displayVenues(data.venues);
    }
  } catch (error) {
    resultsHeading.classList.remove("visible");
    showMessage("Unable to find venues right now.", "Please try again.");
  } finally {
```

```
    submitButton.disabled = false;
    buttonText.textContent = "Suggest Venues";
  }
});
```

PRACTICAL 7

7. Develop backend as required. Also consider location. Prepare project for company meeting, birthday celebration, music concert.

1. server.js

```
const express = require("express");
const path = require("path");
const venues = require("./venues");
const app = express();
const PORT = process.env.PORT || 3000;
const supportedEventTypes = ["Company Meeting", "Birthday Celebration", "Music Concert"];
const supportedLocations = ["Ahmedabad", "Gandhinagar", "Vadodara", "Surat"];
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));
app.get("/api/venues", (req, res) => {
  res.json({ success: true, count: venues.length, venues });
});
app.post("/api/suggest", (req, res) => {
  const { eventType, location, crowdSize } = req.body || {};
  const numericCrowdSize = Number(crowdSize);
  if (!eventType) {
    return res.status(400).json({ success: false, message: "Please select an event type." });
  }
  if (!location) {
    return res.status(400).json({ success: false, message: "Please select a location." });
  }
  if (crowdSize === undefined || crowdSize === null || crowdSize === "") {
    return res.status(400).json({
      success: false,
      message: "Please enter a crowd size."
    });
  }
  if (!Number.isFinite(numericCrowdSize) || numericCrowdSize <= 0) {
    return res.status(400).json({ success: false, message: "Please enter a valid crowd size." });
  }
  if (!supportedEventTypes.includes(eventType)) {
    return res.status(400).json({
      success: false,
      message: "The selected event type is not supported."
    });
  }
  if (!supportedLocations.includes(location)) {
    return res.status(400).json({
      success: false,
      message: "The selected location is not supported."
    });
  }
  const suitableVenues = venues
    .filter((venue) => {
      const supportsEvent = venue.eventTypes.includes(eventType);
      const matchesLocation = venue.location === location;
      const fitsCapacity =
        numericCrowdSize >= venue.minCapacity &&
        numericCrowdSize <= venue.maxCapacity;
      return supportsEvent && matchesLocation && fitsCapacity;
    })
    .sort((firstVenue, secondVenue) => {
      const firstUnusedCapacity = firstVenue.maxCapacity - numericCrowdSize;
      const secondUnusedCapacity = secondVenue.maxCapacity - numericCrowdSize;
      if (firstUnusedCapacity !== secondUnusedCapacity) {
        return firstUnusedCapacity - secondUnusedCapacity;
      }
      return secondVenue.rating - firstVenue.rating;
    });
  if (suitableVenues.length === 0) {
    return res.json({
      success: true,
      count: 0,
      venues: [],
      message: "No exact venue found for your event requirements."
    });
  }
  res.json({
```

```
    success: true,
    count: suitableVenues.length,
    venues: suitableVenues
  });
});
app.use((err, req, res, next) => {
  const malformedJson = err instanceof SyntaxError && err.status === 400;
  if (malformedJson) return res.status(400).json({ success: false, message: "Please send valid JSON." });
  console.error("Request error:", err.message);
  res.status(500).json({ success: false, message: "Unable to process the request right now." });
});
app.listen(PORT, () => {
  console.log(`EVENTORA PRO is running at http://localhost:${PORT}`);
});
```

2. venues.js

```
const venues = [
  {
    id: 1, name: "Executive Crown Hall",
    eventTypes: ["Company Meeting"],
    location: "Ahmedabad",
    minCapacity: 20, maxCapacity: 120,
    priceRange: "Premium",
    rating: 4.8,
    description: "A refined indoor venue designed for focused company meetings."
  },
  {
    id: 2, name: "Heritage Boardroom",
    eventTypes: ["Company Meeting", "Birthday Celebration"],
    location: "Ahmedabad",
    minCapacity: 50, maxCapacity: 180,
    priceRange: "Standard",
    rating: 4.6,
    description: "A flexible city venue with a formal setting and warm ambience."
  },
  {
    id: 3, name: "Capital Grove Pavilion",
    eventTypes: ["Company Meeting", "Birthday Celebration"],
    location: "Gandhinagar",
    minCapacity: 30, maxCapacity: 220,
    priceRange: "Premium",
    rating: 4.7,
    description: "An elegant pavilion suited to business sessions and celebrations."
  },
  {
    id: 4, name: "Golden Courtyard",
    eventTypes: ["Birthday Celebration"],
    location: "Gandhinagar",
    minCapacity: 80, maxCapacity: 300,
    priceRange: "Standard",
    rating: 4.5,
    description: "A spacious celebration hall with an inviting courtyard feel."
  },
  {
    id: 5, name: "Regal Cedar Studio",
    eventTypes: ["Company Meeting", "Birthday Celebration"],
    location: "Vadodara",
    minCapacity: 25, maxCapacity: 160,
    priceRange: "Value",
    rating: 4.4,
    description: "A polished studio for compact professional and social events."
  },
  {
    id: 6, name: "Crescendo Arena",
    eventTypes: ["Music Concert"],
    location: "Vadodara",
    minCapacity: 250, maxCapacity: 650,
    priceRange: "Premium",
    rating: 4.9,
    description: "A large performance venue planned for energetic music concerts."
  },
  {
    id: 7, name: "Pearl Celebration Hall",
    eventTypes: ["Birthday Celebration"],
    location: "Surat",
```

```
    minCapacity: 40, maxCapacity: 140,
    priceRange: "Standard",
    rating: 4.7,
    description: "A graceful hall for memorable birthdays and family gatherings."
  },
  {
    id: 8, name: "Diamond City Forum",
    eventTypes: ["Company Meeting", "Birthday Celebration"],
    location: "Surat",
    minCapacity: 90, maxCapacity: 260,
    priceRange: "Premium",
    rating: 4.6,
    description: "A contemporary forum for larger corporate and private events."
  },
  {
    id: 9, name: "Riverfront Rhythm Grounds",
    eventTypes: ["Music Concert"],
    location: "Ahmedabad",
    minCapacity: 300, maxCapacity: 1000,
    priceRange: "Premium",
    rating: 4.8,
    description: "An open event ground with room for major live performances."
  }
];
module.exports = venues;
```

3. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Find a suitable venue for meetings, birthdays and concerts.">
  <title>EVENTORA PRO | Premium Event Planning</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <div class="container nav-wrap">
      <a class="brand" href="#home" aria-label="EVENTORA PRO home">
        <span class="brand-mark">E</span>
        <span><strong>EVENTORA PRO</strong>
          <small>Plan the right event at the right place.</small></span>
      </a>
      <nav aria-label="Main navigation"><a href="#home">Home</a>
        <a class="nav-cta" href="#planner">Plan Event</a></nav>
    </div>
  </header>
  <main>
    <section class="hero" id="home">
      <div class="container hero-grid">
        <div class="hero-copy">
          <p class="eyebrow">Thoughtful venue recommendations</p>
          <h1>Perfect Events Start with the <em>Perfect Place.</em></h1>
          <p class="hero-text">Choose your event, location and guest count to discover a venue that fits.</p>
          <a class="button button-gold" href="#planner">Plan Your Event</a>
        </div>
        <div class="hero-panel" aria-label="Events we help plan">
          <div class="hero-panel-top"><span>Curated for your occasion</span>
            <span class="gold-line"></span></div>
          <div class="occasion-list">
            <article>
              <span class="occasion-number">01</span>
              <div><h2>Company Meetings</h2>
                <p>Focused, polished and professional.</p></div>
            </article>
            <article>
              <span class="occasion-number">02</span>
              <div><h2>Birthday Celebrations</h2>
                <p>Warm spaces for lasting memories.</p></div>
            </article>
            <article>
              <span class="occasion-number">03</span>
              <div><h2>Music Concerts</h2>
                <p>Large venues made for live energy.</p></div>
```

```
          </article>
        </div>
      </div>
    </div>
  </section>
  <section class="planner-section" id="planner">
    <div class="container">
      <div class="section-heading">
        <p class="eyebrow">Venue finder</p>
        <h2>Plan Your Perfect Event</h2>
        <p>Tell us the essentials. We will match all three requirements.</p>
      </div>
      <form class="finder-form" id="venue-form" novalidate>
        <div class="field-group">
          <label for="event-type">Event Type</label>
          <select id="event-type" name="eventType" required>
            <option value="">Choose an event</option>
            <option value="Company Meeting">Company Meeting</option>
            <option value="Birthday Celebration">Birthday Celebration</option>
            <option value="Music Concert">Music Concert</option>
          </select>
        </div>
        <div class="field-group">
          <label for="location">Location</label>
          <select id="location" name="location" required>
            <option value="">Choose a city</option>
            <option value="Ahmedabad">Ahmedabad</option>
            <option value="Gandhinagar">Gandhinagar</option>
            <option value="Vadodara">Vadodara</option>
            <option value="Surat">Surat</option>
          </select>
        </div>
        <div class="field-group">
          <label for="crowd-size">Crowd Size</label>
          <input id="crowd-size" name="crowdSize" type="number"
            min="1" step="1" placeholder="e.g. 100" required>
        </div>
        <button class="button submit-button" id="submit-button" type="submit">Find Suitable Venue</button>
        <p class="form-message" id="form-message" role="alert" aria-live="polite"></p>
      </form>
    </div>
  </section>
  <section class="results-section" aria-labelledby="results-title">
    <div class="container">
      <div class="results-heading">
        <div><p class="eyebrow">Our recommendation</p>
          <h2 id="results-title">Suitable Venues</h2></div>
        <p class="result-count" id="result-count"></p>
      </div>
      <div class="results-grid" id="results" aria-live="polite">
        <div class="empty-state">
          <span class="empty-symbol">E</span>
          <h3>Your perfect event space is waiting.</h3>
          <p>Complete the planner above to see venues selected for your needs.</p>
        </div>
      </div>
    </div>
  </section>
 </main>
 <footer>
   <div class="container footer-inner">
     <p>EVENTORA PRO</p><p>Plan the right event at the right place.</p>
   </div>
 </footer>
 <script src="app.js"></script>
</body>
</html>
```

4. public/style.css

```
:root {
  --ink: #0c1420; --navy: #111d2b; --navy-light: #192838; --gold: #c9a35f; --gold-light: #e6c987; --cream: #f6f0e5; --paper: #fffdf8; --muted: #aeb6bf;
      --text: #26313d; --border: rgba(201, 163, 95, 0.28); --shadow: 0 24px 55px rgba(5, 13, 22, 0.16);
}
* {
  box-sizing: border-box;
```

```
}
html {
  scroll-behavior: smooth;
}
body {
  margin: 0; color: var(--text); background: var(--paper); font-family: Georgia, "Times New Roman", serif; line-height: 1.6;
}
button,
input,
select {
  font: inherit;
}
a {
  color: inherit; text-decoration: none;
}
.container {
  width: min(1120px, calc(100% - 48px)); margin-inline: auto;
}
.site-header {
  position: absolute; z-index: 10; top: 0; width: 100%; color: var(--cream); border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}
.nav-wrap {
  min-height: 88px; display: flex; align-items: center; justify-content: space-between; gap: 24px;
}
.brand {
  display: flex; align-items: center; gap: 13px;
}
.brand-mark {
  width: 42px; height: 42px; display: grid; place-items: center; color: var(--ink); background: var(--gold); border: 1px solid var(--gold-light); font-size:
    1.25rem; font-weight: 700;
}
.brand strong,
.brand small {
  display: block;
}
.brand strong {
  font-size: 1rem; letter-spacing: 0.16em;
}
.brand small {
  color: var(--muted); font-family: Arial, sans-serif; font-size: 0.69rem; letter-spacing: 0.03em;
}
nav {
  display: flex; align-items: center; gap: 32px; font-family: Arial, sans-serif; font-size: 0.87rem; letter-spacing: 0.08em;
}
nav a {
  transition: color 0.2s ease;
}
nav a:hover {
  color: var(--gold-light);
}
.nav-cta {
  padding: 10px 17px; border: 1px solid var(--gold);
}
.hero {
  min-height: 720px; display: flex; align-items: center; color: var(--cream);
  background:
    radial-gradient(circle at 80% 25%, rgba(201, 163, 95, 0.13), transparent 28%),
    linear-gradient(125deg, #08111c 0%, var(--navy) 58%, #172638 100%);
  overflow: hidden;
}
.hero-grid {
  display: grid; grid-template-columns: 1.15fr 0.85fr; align-items: center; gap: 80px; padding-top: 120px; padding-bottom: 70px;
}
.eyebrow {
  margin: 0 0 14px; color: var(--gold); font-family: Arial, sans-serif; font-size: 0.74rem; font-weight: 700; letter-spacing: 0.22em; text-transform: uppercase;
}
.hero h1 {
  max-width: 680px; margin: 0; font-size: clamp(3rem, 5.4vw, 5.2rem); font-weight: 400; letter-spacing: -0.045em; line-height: 1.02;
}
.hero h1 em {
  color: var(--gold-light); font-weight: 400;
}
.hero-text {
  max-width: 590px; margin: 27px 0 34px; color: #c6ccd2; font-family: Arial, sans-serif; font-size: 1.08rem;
}
```

```
.button {
  display: inline-flex; min-height: 52px; align-items: center; justify-content: center; padding: 0 25px; border: 1px solid transparent; cursor: pointer;
    font-family: Arial, sans-serif; font-size: 0.83rem; font-weight: 700; letter-spacing: 0.08em; transition: transform 0.2s ease, background 0.2s ease;
}
.button:hover {
  transform: translateY(-2px);
}
.button-gold {
  color: var(--ink); background: var(--gold);
}
.button-gold:hover {
  background: var(--gold-light);
}
.hero-panel {
  padding: 32px; border: 1px solid var(--border); background: rgba(255, 255, 255, 0.035); box-shadow: 0 30px 70px rgba(0, 0, 0, 0.22); backdrop-filter:
    blur(5px);
}
.hero-panel-top {
  display: flex; align-items: center; gap: 14px; margin-bottom: 12px; color: var(--gold-light); font-family: Arial, sans-serif; font-size: 0.69rem; letter-spacing:
    0.15em; text-transform: uppercase;
}
.gold-line {
  height: 1px; flex: 1; background: var(--border);
}
.occasion-list article {
  display: grid; grid-template-columns: 42px 1fr; gap: 15px; padding: 25px 0; border-bottom: 1px solid rgba(255, 255, 255, 0.09);
}
.occasion-list article:last-child {
  border-bottom: 0;
}
.occasion-number {
  color: var(--gold); font-family: Arial, sans-serif; font-size: 0.72rem;
}
.occasion-list h2 {
  margin: -6px 0 3px; font-size: 1.35rem; font-weight: 400;
}
.occasion-list p {
  margin: 0; color: var(--muted); font-family: Arial, sans-serif; font-size: 0.83rem;
}
.planner-section {
  padding: 100px 0; background: #f0e9dd;
}
.section-heading {
  max-width: 620px; margin: 0 auto 38px; text-align: center;
}
.section-heading h2,
.results-heading h2 {
  margin: 0; color: var(--ink); font-size: clamp(2.2rem, 4vw, 3.25rem); font-weight: 400; line-height: 1.15;
}
.section-heading > p:last-child {
  margin: 14px 0 0; color: #64707b; font-family: Arial, sans-serif;
}
.finder-form {
  position: relative; display: grid; grid-template-columns: repeat(3, 1fr) auto; align-items: end; gap: 16px; max-width: 1080px; margin: auto; padding: 30px;
    background: var(--paper); border-top: 3px solid var(--gold); box-shadow: var(--shadow);
}
.field-group label {
  display: block; margin-bottom: 8px; color: #515b65; font-family: Arial, sans-serif; font-size: 0.75rem; font-weight: 700; letter-spacing: 0.08em;
    text-transform: uppercase;
}
.field-group select,
.field-group input {
  width: 100%; height: 52px; padding: 0 14px; color: var(--text); background: #fff; border: 1px solid #d6d1c9; border-radius: 0; outline: none; font-family:
    Arial, sans-serif;
}
.field-group select:focus,
.field-group input:focus {
  border-color: var(--gold); box-shadow: 0 0 0 3px rgba(201, 163, 95, 0.13);
}
.submit-button {
  color: var(--cream); background: var(--navy); white-space: nowrap;
}
.submit-button:hover {
  background: #1b2c3e;
}
```

```
.submit-button:disabled {
  cursor: wait; opacity: 0.68; transform: none;
}
.form-message {
  min-height: 20px; grid-column: 1 / -1; margin: 0; color: #9b2f2f; font-family: Arial, sans-serif; font-size: 0.85rem;
}
.results-section {
  min-height: 500px; padding: 95px 0 110px; background: var(--paper);
}
.results-heading {
  display: flex; align-items: end; justify-content: space-between; gap: 24px; margin-bottom: 34px;
}
.result-count {
  margin: 0 0 8px; color: #69737c; font-family: Arial, sans-serif; font-size: 0.83rem;
}
.results-grid {
  display: grid; grid-template-columns: repeat(3, 1fr); gap: 22px;
}
.empty-state {
  grid-column: 1 / -1; padding: 58px 24px; text-align: center; border: 1px dashed #cfc8bb; background: #fbf8f1;
}
.empty-symbol {
  width: 50px; height: 50px; display: grid; place-items: center; margin: 0 auto 17px; color: var(--ink); border: 1px solid var(--gold); font-size: 1.4rem;
}
.empty-state h3 {
  margin: 0 0 6px; color: var(--ink); font-size: 1.45rem; font-weight: 400;
}
.empty-state p {
  margin: 0; color: #707981; font-family: Arial, sans-serif; font-size: 0.9rem;
}
.venue-card {
  position: relative; padding: 27px; overflow: hidden; border: 1px solid #ddd7cc; background: #fff; box-shadow: 0 12px 30px rgba(12, 20, 32, 0.08);
}
.venue-card::before {
  position: absolute; top: 0; left: 0; width: 100%; height: 3px; background: var(--gold); content: "";
}
.venue-topline {
  display: flex; align-items: center; justify-content: space-between; gap: 15px; color: #737b82; font-family: Arial, sans-serif; font-size: 0.75rem;
}
.rating {
  color: #80601e; font-weight: 700;
}
.venue-card h3 {
  margin: 17px 0 11px; color: var(--ink); font-size: 1.55rem; font-weight: 400; line-height: 1.2;
}
.venue-description {
  min-height: 54px; margin: 0 0 20px; color: #66717a; font-family: Arial, sans-serif; font-size: 0.86rem;
}
.venue-details {
  display: grid; gap: 9px; padding-top: 17px; border-top: 1px solid #e6e1d8;
}
.venue-detail {
  display: flex; justify-content: space-between; gap: 18px; font-family: Arial, sans-serif; font-size: 0.8rem;
}
.venue-detail span:first-child {
  color: #7a8289;
}
.venue-detail span:last-child {
  color: var(--ink); font-weight: 700; text-align: right;
}
footer {
  color: #aeb6bf; background: #08111c; font-family: Arial, sans-serif; font-size: 0.8rem;
}
.footer-inner {
  min-height: 92px; display: flex; align-items: center; justify-content: space-between; gap: 20px;
}
.footer-inner p {
  margin: 0;
}
.footer-inner p:first-child {
  color: var(--gold-light); font-family: Georgia, serif; letter-spacing: 0.14em;
}
@media (max-width: 900px) {
  .hero-grid {
    grid-template-columns: 1fr; gap: 55px; padding-top: 160px;
```

```
  }
  .hero-panel {
    max-width: 640px;
  }
  .finder-form {
    grid-template-columns: 1fr 1fr;
  }
  .submit-button {
    width: 100%;
  }
  .results-grid {
    grid-template-columns: 1fr 1fr;
  }
}
@media (max-width: 600px) {
  .container {
    width: min(100% - 30px, 1120px);
  }
  .nav-wrap {
    min-height: 78px;
  }
  .brand small {
    display: none;
  }
  .brand strong {
    font-size: 0.83rem;
  }
  .brand-mark {
    width: 36px; height: 36px;
  }
  nav {
    gap: 14px;
  }
  nav > a:first-child {
    display: none;
  }
  .nav-cta {
    padding: 8px 11px;
  }
  .hero {
    min-height: auto;
  }
  .hero-grid {
    gap: 44px; padding-top: 135px; padding-bottom: 60px;
  }
  .hero h1 {
    font-size: clamp(2.65rem, 14vw, 4rem);
  }
  .hero-panel {
    padding: 22px;
  }
  .planner-section,
  .results-section {
    padding: 72px 0;
  }
  .finder-form {
    grid-template-columns: 1fr; padding: 24px 20px;
  }
  .form-message {
    grid-column: 1;
  }
  .results-heading {
    align-items: start; flex-direction: column; gap: 8px;
  }
  .results-grid {
    grid-template-columns: 1fr;
  }
  .footer-inner {
    align-items: start; flex-direction: column; justify-content: center; padding-block: 24px;
  }
}
```

5. public/app.js

```
const form = document.getElementById("venue-form");
const eventTypeInput = document.getElementById("event-type");
const locationInput = document.getElementById("location");
const crowdSizeInput = document.getElementById("crowd-size");
const submitButton = document.getElementById("submit-button");
const formMessage = document.getElementById("form-message");
const results = document.getElementById("results");
const resultCount = document.getElementById("result-count");
function showEmptyState(title, message) {
  results.replaceChildren();
  const emptyState = document.createElement("div");
  emptyState.className = "empty-state";
  const symbol = document.createElement("span");
  symbol.className = "empty-symbol";
  symbol.textContent = "E";
  const heading = document.createElement("h3");
  heading.textContent = title;
  const paragraph = document.createElement("p");
  paragraph.textContent = message;
  emptyState.append(symbol, heading, paragraph);
  results.appendChild(emptyState);
}
function addDetail(container, label, value) {
  const row = document.createElement("div");
  row.className = "venue-detail";
  const detailLabel = document.createElement("span");
  detailLabel.textContent = label;
  const detailValue = document.createElement("span");
  detailValue.textContent = value;
  row.append(detailLabel, detailValue);
  container.appendChild(row);
}
function createVenueCard(venue) {
  const card = document.createElement("article");
  card.className = "venue-card";
  const topLine = document.createElement("div");
  topLine.className = "venue-topline";
  const location = document.createElement("span");
  location.textContent = venue.location;
  const rating = document.createElement("span");
  rating.className = "rating";
  rating.textContent = `★ ${venue.rating}`;
  const name = document.createElement("h3");
  name.textContent = venue.name;
  const description = document.createElement("p");
  description.className = "venue-description";
  description.textContent = venue.description;
  const details = document.createElement("div");
  details.className = "venue-details";
  addDetail(details, "Suitable for", venue.eventTypes.join(", "));
  addDetail(details, "Capacity", `${venue.minCapacity}–${venue.maxCapacity} guests`);
  addDetail(details, "Price range", venue.priceRange);
  topLine.append(location, rating);
  card.append(topLine, name, description, details);
  return card;
}
function displayVenues(venues) {
  results.replaceChildren();
  resultCount.textContent = `${venues.length} suitable venue${venues.length === 1 ? "" : "s"}`;
  venues.forEach((venue) => {
    results.appendChild(createVenueCard(venue));
  });
}
function validateForm(eventType, location, crowdSize) {
  if (!eventType) {
    return "Please select an event type.";
  }
  if (!location) {
    return "Please select a location.";
  }
  if (!Number.isFinite(crowdSize) || crowdSize <= 0) {
    return "Please enter a valid crowd size.";
  }
  return "";
```

```
}
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const eventType = eventTypeInput.value;
  const location = locationInput.value;
  const crowdSize = Number(crowdSizeInput.value);
  const validationMessage = validateForm(eventType, location, crowdSize);
  formMessage.textContent = validationMessage;
  if (validationMessage) {
    return;
  }
  submitButton.disabled = true;
  submitButton.textContent = "Finding Venue...";
  formMessage.textContent = "";
  resultCount.textContent = "Searching...";
  try {
    const response = await fetch("/api/suggest", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ eventType, location, crowdSize })
    });
    const data = await response.json();
    if (!response.ok) {
      formMessage.textContent = data.message || "Please check your event details.";
      resultCount.textContent = "";
      return;
    }
    if (data.count === 0) {
      resultCount.textContent = "0 suitable venues";
      showEmptyState(
        "No exact venue found.",
        "Try changing the location, event type or guest count."
      );
      return;
    }
    displayVenues(data.venues);
    document.querySelector(".results-section").scrollIntoView({ behavior: "smooth" });
  } catch (error) {
    resultCount.textContent = "";
    showEmptyState(
      "Unable to find venues right now.",
      "Please try again."
    );
  } finally {
    submitButton.disabled = false;
    submitButton.textContent = "Find Suitable Venue";
  }
});
```

PRACTICAL 8

8. Build a robust Todo List App with ReactJS

1. src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './App.css'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

2. src/App.jsx

```
import { useEffect, useState } from 'react'
import TodoForm from './components/TodoForm.jsx'
import TodoList from './components/TodoList.jsx'

const STORAGE_KEY = 'taskflow-todos'
const FILTERS = ['All', 'Active', 'Completed']

function loadTodos() {
  try {
    const saved = JSON.parse(localStorage.getItem(STORAGE_KEY))
    return Array.isArray(saved) ? saved : []
  } catch {
    return []
  }
}

function App() {
  const [todos, setTodos] = useState(loadTodos)
  const [filter, setFilter] = useState('All')

  useEffect(() => {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(todos))
  }, [todos])

  const addTodo = (text) => {
    setTodos((current) => [
      ...current,
      { id: Date.now(), text, completed: false },
    ])
  }

  const toggleTodo = (id) => {
    setTodos((current) =>
      current.map((todo) =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo,
      ),
    )
  }

  const updateTodo = (id, text) => {
    setTodos((current) =>
      current.map((todo) => (todo.id === id ? { ...todo, text } : todo)),
    )
  }

  const deleteTodo = (id) => {
    setTodos((current) => current.filter((todo) => todo.id !== id))
  }

  const clearCompleted = () => {
    setTodos((current) => current.filter((todo) => !todo.completed))
  }

  const filteredTodos = todos.filter((todo) => {
    if (filter === 'Active') return !todo.completed
    if (filter === 'Completed') return todo.completed
    return true
  })
```

```
  const activeCount = todos.filter((todo) => !todo.completed).length
  const hasCompleted = todos.some((todo) => todo.completed)

  return (
    <main className="page-shell">
      <section className="task-app" aria-labelledby="app-title">
        <header className="app-header">
          <div className="brand-mark" aria-hidden="true">
            <span />
          </div>
          <div>
            <p className="eyebrow">Daily focus</p>
            <h1 id="app-title">TaskFlow</h1>
            <p className="tagline">Organize today. Accomplish more.</p>
          </div>
        </header>

        <TodoForm onAdd={addTodo} />

        <div className="task-summary">
          <div className="filters" aria-label="Filter tasks">
            {FILTERS.map((name) => (
              <button
                className={filter === name ? 'filter-button active' : 'filter-button'}
                key={name}
                onClick={() => setFilter(name)}
                type="button"
                aria-pressed={filter === name}
              >
                {name}
              </button>
            ))}
          </div>
          <p className="task-count" aria-live="polite">
            <strong>{activeCount}</strong> {activeCount === 1 ? 'task' : 'tasks'} left
          </p>
        </div>

        <TodoList
          todos={filteredTodos}
          filter={filter}
          onToggle={toggleTodo}
          onUpdate={updateTodo}
          onDelete={deleteTodo}
        />

        <footer className="app-footer">
          <p>{todos.length ? `${todos.length} total` : 'Your list is clear'}</p>
          <button
            className="clear-button"
            type="button"
            onClick={clearCompleted}
            disabled={!hasCompleted}
          >
            Clear Completed
          </button>
        </footer>
      </section>
      <p className="page-note">Small steps, steady progress.</p>
    </main>
  )
}

export default App
```

3. src/components/TodoForm.jsx

```
import { useState } from 'react'

function TodoForm({ onAdd }) {
  const [text, setText] = useState('')
  const [error, setError] = useState('')
  const handleSubmit = (event) => {
    event.preventDefault()
    const cleanText = text.trim()
    if (!cleanText) {
```

```
      setError('Please enter a task before adding it.')
      return
    }

    onAdd(cleanText)
    setText('')
    setError('')
  }

  const handleChange = (event) => {
    setText(event.target.value)
    if (error) setError('')
  }

  return (
    <form className="todo-form" onSubmit={handleSubmit} noValidate>
      <label htmlFor="new-todo">Add a new task</label>
      <div className="form-row">
        <input
          id="new-todo"
          value={text}
          onChange={handleChange}
          placeholder="What needs to be done?"
          autoComplete="off"
          aria-describedby={error ? 'todo-error' : undefined}
          aria-invalid={Boolean(error)}
        />
        <button type="submit">Add Task</button>
      </div>
      {error && (
        <p className="form-error" id="todo-error" role="alert">
          {error}
        </p>
      )}
    </form>
  )
}

export default TodoForm
```

4. src/components/TodoItem.jsx

```
import { useState } from 'react'

function TodoItem({ todo, onToggle, onUpdate, onDelete }) {
  const [isEditing, setIsEditing] = useState(false)
  const [editText, setEditText] = useState(todo.text)
  const [error, setError] = useState('')

  const saveEdit = () => {
    const cleanText = editText.trim()

    if (!cleanText) {
      setError('Task text cannot be empty.')
      return
    }

    onUpdate(todo.id, cleanText)
    setIsEditing(false)
    setError('')
  }

  const cancelEdit = () => {
    setEditText(todo.text)
    setIsEditing(false)
    setError('')
  }

  const handleKeyDown = (event) => {
    if (event.key === 'Enter') saveEdit()
    if (event.key === 'Escape') cancelEdit()
  }

  if (isEditing) {
    return (
      <li className="todo-item editing">
```

```
        <div className="edit-area">
          <label className="sr-only" htmlFor={`edit-${todo.id}`}>
            Edit task
          </label>
          <input
            id={`edit-${todo.id}`}
            className="edit-input"
            value={editText}
            onChange={(event) => {
              setEditText(event.target.value)
              if (error) setError('')
            }}
            onKeyDown={handleKeyDown}
            aria-invalid={Boolean(error)}
            aria-describedby={error ? `edit-error-${todo.id}` : undefined}
            autoFocus
          />
          {error && (
            <p className="edit-error" id={`edit-error-${todo.id}`} role="alert">
              {error}
            </p>
          )}
        </div>
        <div className="item-actions">
          <button className="save-button" type="button" onClick={saveEdit}>Save</button>
          <button className="text-button" type="button" onClick={cancelEdit}>Cancel</button>
        </div>
      </li>
    )
  }

  return (
    <li className={todo.completed ? 'todo-item completed' : 'todo-item'}>
      <label className="check-control">
        <input
          type="checkbox"
          checked={todo.completed}
          onChange={() => onToggle(todo.id)}
          aria-label={`Mark ${todo.text} as ${todo.completed ? 'active' : 'completed'}`}
        />
        <span className="custom-check" aria-hidden="true">✓</span>
      </label>
      <span className="todo-text">{todo.text}</span>
      <div className="item-actions">
        <button className="text-button" type="button" onClick={() => setIsEditing(true)}>
          Edit
        </button>
        <button className="delete-button" type="button" onClick={() => onDelete(todo.id)}>
          Delete
        </button>
      </div>
    </li>
  )
}

export default TodoItem
```

5. src/components/TodoList.jsx

```
import TodoItem from './TodoItem.jsx'

const emptyMessages = {
  All: ['No tasks yet.', 'Add your first task to get started.'],
  Active: ['No active tasks.', 'You are all caught up.'],
  Completed: ['No completed tasks.', 'Finished tasks will appear here.'],
}

function TodoList({ todos, filter, onToggle, onUpdate, onDelete }) {
  if (!todos.length) {
    const [title, message] = emptyMessages[filter]

    return (
      <div className="empty-state" role="status">
        <div className="empty-icon" aria-hidden="true">✓</div>
        <h2>{title}</h2>
        <p>{message}</p>
```

```
      </div>
    )
  }

  return (
    <ul className="todo-list" aria-label={`${filter} tasks`}>
      {todos.map((todo) => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onToggle={onToggle}
          onUpdate={onUpdate}
          onDelete={onDelete}
        />
      ))}
    </ul>
  )
}

export default TodoList
```

6. src/App.css

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #202235;
  background: #f3f4f8;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}

button, input { font: inherit; }
button { cursor: pointer; }
button:disabled { cursor: not-allowed; }

.page-shell {
  min-height: 100vh;
  padding: 64px 24px 32px;
  background:
    radial-gradient(circle at 15% 15%, rgba(91, 91, 214, 0.1), transparent 28rem),
    linear-gradient(180deg, #f8f8fc 0%, #eef0f6 100%);
}

.task-app {
  width: min(100%, 780px);
  margin: 0 auto;
  overflow: hidden;
  background: rgba(255, 255, 255, 0.96);
  border: 1px solid #e4e5ed;
  border-radius: 20px;
  box-shadow: 0 24px 70px rgba(38, 41, 70, 0.12);
}

.app-header {
  display: flex;
  align-items: center;
  gap: 18px;
  padding: 32px 36px 24px;
}

.brand-mark {
  display: grid;
  place-items: center;
  flex: 0 0 54px;
  width: 54px;
  height: 54px;
  border-radius: 16px;
  background: #5b5bd6;
```

```
  box-shadow: 0 10px 24px rgba(91, 91, 214, 0.3);
}

.brand-mark span {
  width: 24px;
  height: 12px;
  border: solid #fff;
  border-width: 0 0 4px 4px;
  transform: rotate(-45deg) translate(2px, -2px);
}

.eyebrow {
  margin: 0 0 4px;
  color: #5b5bd6;
  font-size: 0.75rem;
  font-weight: 800;
  letter-spacing: 0.14em;
  text-transform: uppercase;
}

h1 {
  margin: 0;
  font-size: clamp(2rem, 6vw, 2.75rem);
  line-height: 1.05;
  letter-spacing: -0.04em;
}

.tagline {
  margin: 8px 0 0;
  color: #6b6e83;
  font-size: 1rem;
}

.todo-form { padding: 0 36px 28px; }

.todo-form > label {
  display: block;
  margin-bottom: 9px;
  color: #494c61;
  font-size: 0.875rem;
  font-weight: 700;
}

.form-row {
  display: flex;
  gap: 10px;
}

.form-row input, .edit-input {
  min-width: 0;
  width: 100%;
  padding: 13px 15px;
  color: #24263a;
  background: #fafaff;
  border: 1px solid #d9dbe6;
  border-radius: 10px;
  outline: none;
  transition: border-color 150ms ease, box-shadow 150ms ease;
}

.form-row input::placeholder { color: #989bad; }
.form-row input:focus, .edit-input:focus {
  border-color: #5b5bd6;
  box-shadow: 0 0 0 4px rgba(91, 91, 214, 0.13);
}
.form-row input[aria-invalid="true"],
.edit-input[aria-invalid="true"] { border-color: #c74755; }

.form-row button, .save-button {
  flex: 0 0 auto;
  padding: 12px 20px;
  color: #fff;
  font-weight: 750;
  background: #5b5bd6;
  border: 0;
```

```
  border-radius: 10px;
  transition: background 150ms ease, transform 150ms ease;
}

.form-row button:hover, .save-button:hover { background: #4949bf; }
.form-row button:active, .save-button:active { transform: translateY(1px); }
.form-error, .edit-error {
  margin: 8px 2px 0;
  color: #b52c3b;
  font-size: 0.875rem;
}

.task-summary {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 18px;
  padding: 16px 36px;
  background: #f7f7fb;
  border-block: 1px solid #e7e8ef;
}

.filters {
  display: flex;
  gap: 4px;
  padding: 4px;
  background: #e9eaf2;
  border-radius: 10px;
}

.filter-button {
  padding: 7px 12px;
  color: #5d6073;
  font-size: 0.875rem;
  font-weight: 700;
  background: transparent;
  border: 0;
  border-radius: 7px;
}

.filter-button:hover { color: #303246; }
.filter-button.active {
  color: #3f3fac;
  background: #fff;
  box-shadow: 0 2px 8px rgba(41, 44, 75, 0.1);
}

.task-count {
  flex: 0 0 auto;
  margin: 0;
  color: #707386;
  font-size: 0.9rem;
}
.task-count strong { color: #303246; }

.todo-list {
  display: grid;
  gap: 1px;
  margin: 0;
  padding: 0;
  list-style: none;
  background: #ececf2;
}

.todo-item {
  display: grid;
  grid-template-columns: 24px minmax(0, 1fr) auto;
  align-items: center;
  gap: 13px;
  min-height: 68px;
  padding: 13px 36px;
  background: #fff;
  transition: background 150ms ease;
}
.todo-item:hover { background: #fdfdff; }
```

```
.todo-item.completed { background: #fafafa; }
.check-control {
  position: relative;
  display: grid;
  place-items: center;
  width: 22px;
  height: 22px;
}

.check-control input {
  position: absolute;
  width: 100%;
  height: 100%;
  margin: 0;
  opacity: 0;
  cursor: pointer;
}

.custom-check {
  display: grid;
  place-items: center;
  width: 20px;
  height: 20px;
  color: transparent;
  font-size: 0.75rem;
  font-weight: 900;
  border: 2px solid #b6b8c6;
  border-radius: 6px;
}

.check-control input:checked + .custom-check {
  color: #fff;
  background: #5b5bd6;
  border-color: #5b5bd6;
}

.check-control input:focus-visible + .custom-check {
  box-shadow: 0 0 0 4px rgba(91, 91, 214, 0.18);
}

.todo-text {
  min-width: 0;
  overflow-wrap: anywhere;
  line-height: 1.5;
}

.completed .todo-text {
  color: #777a8a;
  text-decoration: line-through 2px;
}

.item-actions {
  display: flex;
  align-items: center;
  gap: 4px;
}

.text-button, .delete-button, .clear-button {
  padding: 8px 10px;
  font-size: 0.85rem;
  font-weight: 700;
  background: transparent;
  border: 0;
  border-radius: 7px;
}

.text-button { color: #51546a; }
.text-button:hover { color: #3f3fac; background: #eeeeff; }
.delete-button { color: #b52c3b; }
.delete-button:hover { background: #fff0f1; }
.todo-item.editing { grid-template-columns: minmax(0, 1fr) auto; }
.edit-area { min-width: 0; }
.save-button { padding: 9px 14px; font-size: 0.85rem; }

.empty-state {
```

```
  padding: 64px 24px;
  text-align: center;
}

.empty-icon {
  display: grid;
  place-items: center;
  width: 44px;
  height: 44px;
  margin: 0 auto 15px;
  color: #5b5bd6;
  font-size: 1.25rem;
  font-weight: 900;
  background: #ededff;
  border-radius: 50%;
}

.empty-state h2 { margin: 0; font-size: 1.1rem; }
.empty-state p { margin: 7px 0 0; color: #76798c; font-size: 0.95rem; }
.app-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  min-height: 64px;
  padding: 12px 36px;
  border-top: 1px solid #e7e8ef;
}

.app-footer p { margin: 0; color: #8a8d9c; font-size: 0.85rem; }
.clear-button { color: #55586c; border: 1px solid #dfe0e8; }
.clear-button:hover:not(:disabled) {
  color: #b52c3b;
  background: #fff5f5;
  border-color: #f1cfd3;
}
.clear-button:disabled { color: #aeb0bd; background: #f7f7f9; }
.page-note {
  margin: 20px auto 0;
  color: #8a8d9d;
  font-size: 0.8rem;
  text-align: center;
}

button:focus-visible {
  outline: 3px solid rgba(91, 91, 214, 0.35);
  outline-offset: 2px;
}

.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

@media (max-width: 600px) {
  .page-shell { padding: 20px 12px; }
  .task-app { border-radius: 16px; }
  .app-header, .todo-form, .todo-item, .task-summary, .app-footer { padding-inline: 18px; }
  .app-header { align-items: flex-start; padding-top: 24px; }
  .brand-mark {
    flex-basis: 46px;
    width: 46px;
    height: 46px;
    border-radius: 13px;
  }
  .form-row { flex-direction: column; }
  .form-row button { width: 100%; }
  .task-summary { align-items: flex-start; flex-direction: column; gap: 10px; }
```

```
  .filters { width: 100%; }
  .filter-button { flex: 1; padding-inline: 6px; }
  .todo-item {
    grid-template-columns: 24px minmax(0, 1fr);
    row-gap: 8px;
    padding-block: 15px;
  }
  .todo-item > .item-actions { grid-column: 2; justify-content: flex-end; }
  .todo-item.editing { grid-template-columns: minmax(0, 1fr); }
  .todo-item.editing > .item-actions { grid-column: 1; }
  .empty-state { padding-block: 48px; }
}

@media (prefers-reduced-motion: reduce) {
  * { scroll-behavior: auto !important; transition: none !important; }
}
```

PRACTICAL 9

9. Write a program to create a voting application using React JS

1. src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import "./App.css";
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

2. src/App.jsx

```
import { useEffect, useState } from "react";
import VotingPanel from "./components/VotingPanel";
import ResultsPanel from "./components/ResultsPanel";

const STORAGE_KEY = "votesphere-candidates";
const defaultCandidates = [
  { id: 1, name: "Aarav Shah", group: "Unity Group", votes: 0 },
  { id: 2, name: "Meera Patel", group: "Progress Group", votes: 0 },
  { id: 3, name: "Kabir Mehta", group: "Vision Group", votes: 0 },
  { id: 4, name: "Riya Desai", group: "Future Group", votes: 0 },
];

function loadCandidates() {
  try {
    const saved = JSON.parse(localStorage.getItem(STORAGE_KEY));

    if (!Array.isArray(saved)) return defaultCandidates;

    return defaultCandidates.map((candidate) => {
      const storedCandidate = saved.find((item) => item.id === candidate.id);
      const savedVotes = storedCandidate?.votes;

      return {
        ...candidate,
        votes:
          Number.isInteger(savedVotes) && savedVotes >= 0
            ? savedVotes
            : candidate.votes,
      };
    });
  } catch {
    return defaultCandidates;
  }
}

function App() {
  const [candidates, setCandidates] = useState(loadCandidates);
  const [selectedId, setSelectedId] = useState(null);
  const [feedback, setFeedback] = useState({ type: "", text: "" });

  useEffect(() => {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(candidates));
  }, [candidates]);

  const totalVotes = candidates.reduce(
    (total, candidate) => total + candidate.votes,
    0,
  );

  function handleSelect(candidateId) {
    setSelectedId(candidateId);
    setFeedback({ type: "", text: "" });
  }

  function handleVote() {
    if (selectedId === null) {
      setFeedback({
        type: "error",
```

```
        text: "Please select a candidate before voting.",
      });
      return;
    }

    setCandidates((currentCandidates) =>
      currentCandidates.map((candidate) =>
        candidate.id === selectedId
          ? { ...candidate, votes: candidate.votes + 1 }
          : candidate,
      ),
    );
    setSelectedId(null);
    setFeedback({ type: "success", text: "Your vote has been recorded." });
  }

  function handleReset() {
    const shouldReset = window.confirm(
      "Reset all VoteSphere election results?",
    );

    if (!shouldReset) return;

    setCandidates(defaultCandidates.map((candidate) => ({ ...candidate })));
    setSelectedId(null);
    setFeedback({
      type: "success",
      text: "Election results have been reset.",
    });
  }

  return (
    <main className="page-shell">
      <header className="hero">
        <div className="brand-row">
          <span className="brand-mark" aria-hidden="true">V</span>
          <span className="brand-name">VoteSphere</span>
          <span className="demo-badge">Demo</span>
        </div>
        <p className="eyebrow">Campus voices, counted clearly</p>
        <h1>Student Council Election</h1>
        <p className="hero-copy">Choose one candidate and cast your vote.</p>
      </header>

      <section className="workspace" aria-label="Voting workspace">
        <VotingPanel
          candidates={candidates}
          selectedId={selectedId}
          feedback={feedback}
          onSelect={handleSelect}
          onVote={handleVote}
        />

        <ResultsPanel candidates={candidates} totalVotes={totalVotes} />
      </section>

      <footer className="page-footer">
        <div>
          <strong>Classroom demonstration</strong>
          <p>Votes are saved only in this browser using localStorage.</p>
        </div>
        <button className="reset-button" type="button" onClick={handleReset}>
          Reset Election
        </button>
      </footer>
    </main>
  );
}

export default App;
```

3. src/components/CandidateCard.jsx

```
function CandidateCard({ candidate, isSelected, onSelect }) {
  const initials = candidate.name
    .split(" ")
    .map((part) => part[0])
    .join("");
  return (
    <label className={`candidate-card${isSelected ? " selected" : ""}`}>
      <input
        type="radio"
        name="candidate"
        value={candidate.id}
        checked={isSelected}
        onChange={() => onSelect(candidate.id)}
      />
      <span className="candidate-avatar" aria-hidden="true">{initials}</span>
      <span className="candidate-details">
        <strong>{candidate.name}</strong>
        <span>{candidate.group}</span>
      </span>
      <span className="radio-visual" aria-hidden="true" />
      <span className="selection-text">
        {isSelected ? "Selected" : "Select"}
      </span>
    </label>
  );
}

export default CandidateCard;
```

4. src/components/VotingPanel.jsx

```
import CandidateCard from "./CandidateCard";

function VotingPanel({
  candidates,
  selectedId,
  feedback,
  onSelect,
  onVote,
}) {
  return (
    <section className="panel voting-panel" aria-labelledby="candidate-heading">
      <div className="section-heading">
        <span className="step-number">01</span>
        <div>
          <h2 id="candidate-heading">Candidate Selection</h2>
          <p>Select one of the four fictional candidates.</p>
        </div>
      </div>

      <div className="candidate-grid">
        {candidates.map((candidate) => (
          <CandidateCard
            key={candidate.id}
            candidate={candidate}
            isSelected={candidate.id === selectedId}
            onSelect={onSelect}
          />
        ))}
      </div>

      <div className="vote-actions">
        <button className="vote-button" type="button" onClick={onVote}>
          Cast Vote
          <span aria-hidden="true">→</span>
        </button>
        <p
          className={`feedback ${feedback.type}`}
          role={feedback.type === "error" ? "alert" : "status"}
          aria-live="polite"
        >
          {feedback.text || "Your selection stays private on this demo screen."}
        </p>
      </div>
```

```
    </section>
  );
}

export default VotingPanel;
```

5. src/components/ResultsPanel.jsx

```
function ResultsPanel({ candidates, totalVotes }) {
  const highestVotes = Math.max(...candidates.map((candidate) => candidate.votes));
  const leaders = candidates.filter(
    (candidate) => candidate.votes === highestVotes,
  );
  let resultSummary = "No votes cast yet.";
  if (totalVotes > 0 && leaders.length === 1) {
    resultSummary = `Currently Leading: ${leaders[0].name}`;
  } else if (totalVotes > 0) {
    resultSummary = "Current Result: Tie";
  }

  return (
    <section className="panel results-panel" aria-labelledby="results-heading">
      <div className="results-topline">
        <div className="section-heading">
          <span className="step-number teal">02</span>
          <div>
            <h2 id="results-heading">Live Results</h2>
            <p>Updates instantly after every vote.</p>
          </div>
        </div>
        <div className="total-card">
          <span>Total Votes</span>
          <strong>{totalVotes}</strong>
        </div>
      </div>

      <div className="result-list">
        {candidates.map((candidate) => {
          const percentage =
            totalVotes === 0
              ? 0
              : Math.round((candidate.votes / totalVotes) * 100);

          return (
            <div className="result-item" key={candidate.id}>
              <div className="result-labels">
                <div>
                  <strong>{candidate.name}</strong>
                  <span>{candidate.votes} {candidate.votes === 1 ? "vote" : "votes"}</span>
                </div>
                <strong className="percentage">{percentage}%</strong>
              </div>
              <div
                className="progress-track"
                role="progressbar"
                aria-label={`${candidate.name}: ${percentage}%`}
                aria-valuemin="0"
                aria-valuemax="100"
                aria-valuenow={percentage}
              >
                <span className="progress-fill" style={{ width: `${percentage}%` }} />
              </div>
            </div>
          );
        })}
      </div>

      <p className="leader-summary" aria-live="polite">
        <span aria-hidden="true">●</span>
        {resultSummary}
      </p>
    </section>
  );
}

export default ResultsPanel;
```

6. src/App.css

```
:root {
  font-family: "Segoe UI", Arial, sans-serif;
  color: #17213c;
  background: #f3f5fa;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
  background:
    radial-gradient(circle at 88% 6%, rgba(53, 181, 166, 0.12), transparent 24rem),
    #f3f5fa;
}

button, input { font: inherit; }

button:focus-visible,
input:focus-visible + .candidate-avatar {
  outline: 3px solid #91a7ff;
  outline-offset: 3px;
}

.page-shell {
  width: min(1120px, calc(100% - 40px));
  margin: 0 auto;
  padding: 42px 0 28px;
}

.hero {
  position: relative;
  overflow: hidden;
  padding: 32px 36px 34px;
  color: #fff;
  background: #293d91;
  border-radius: 24px 24px 8px 8px;
}

.hero::after {
  position: absolute;
  right: -65px;
  bottom: -105px;
  width: 280px;
  height: 280px;
  content: "";
  border: 45px solid rgba(255, 255, 255, 0.08);
  border-radius: 50%;
}

.brand-row {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 32px;
}

.brand-mark {
  display: grid;
  width: 34px;
  height: 34px;
  place-items: center;
  font-size: 17px;
  font-weight: 800;
  color: #293d91;
  background: #fff;
  border-radius: 10px;
}

.brand-name, h1, h2 { font-family: "Trebuchet MS", "Segoe UI", sans-serif; }
```

```
.brand-name {
  font-size: 1.15rem;
  font-weight: 800;
  letter-spacing: -0.02em;
}

.demo-badge {
  padding: 4px 9px;
  margin-left: 4px;
  font-size: 0.7rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.09em;
  background: rgba(255, 255, 255, 0.15);
  border: 1px solid rgba(255, 255, 255, 0.24);
  border-radius: 99px;
}

.eyebrow {
  margin: 0 0 8px;
  font-size: 0.75rem;
  font-weight: 700;
  color: #8fe0d5;
  text-transform: uppercase;
  letter-spacing: 0.14em;
}

h1 {
  max-width: 650px;
  margin: 0;
  font-size: clamp(2rem, 5vw, 3.25rem);
  line-height: 1.08;
  letter-spacing: -0.045em;
}

.hero-copy { margin: 12px 0 0; color: #dbe1ff; font-size: 1rem; }

.workspace {
  display: grid;
  grid-template-columns: minmax(0, 1.08fr) minmax(360px, 0.92fr);
  background: #fff;
  border: 1px solid #e3e7f0;
  border-top: 0;
  box-shadow: 0 24px 60px rgba(31, 45, 91, 0.11);
}

.panel { min-width: 0; padding: 34px; }
.voting-panel { border-right: 1px solid #e6e9f1; }

.section-heading {
  display: flex;
  align-items: flex-start;
  gap: 13px;
}

.step-number {
  display: grid;
  flex: 0 0 auto;
  width: 34px;
  height: 34px;
  place-items: center;
  font-size: 0.72rem;
  font-weight: 800;
  color: #fff;
  background: #3a50ad;
  border-radius: 9px;
}

.step-number.teal { background: #168b82; }

h2 {
  margin: 0;
  color: #1a2441;
  font-size: 1.25rem;
  letter-spacing: -0.025em;
```

```
}

.section-heading p { margin: 4px 0 0; color: #707891; font-size: 0.83rem; }

.candidate-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
  margin-top: 26px;
}

.candidate-card {
  position: relative;
  display: grid;
  grid-template-columns: auto minmax(0, 1fr) auto;
  align-items: center;
  gap: 12px;
  min-height: 90px;
  padding: 15px;
  cursor: pointer;
  background: #fafbfe;
  border: 1px solid #dde2ed;
  border-radius: 14px;
  transition: border-color 160ms ease, box-shadow 160ms ease, transform 160ms ease;
}

.candidate-card:hover { border-color: #9ca9dc; transform: translateY(-1px); }

.candidate-card.selected {
  background: #f2f5ff;
  border-color: #5269c9;
  box-shadow: 0 0 0 2px rgba(82, 105, 201, 0.12);
}

.candidate-card input { position: absolute; opacity: 0; }

.candidate-avatar {
  display: grid;
  width: 42px;
  height: 42px;
  place-items: center;
  color: #31489f;
  font-weight: 800;
  background: #e7ebfb;
  border-radius: 12px;
}

.candidate-details { display: flex; min-width: 0; flex-direction: column; }

.candidate-details strong {
  overflow: hidden;
  font-size: 0.92rem;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.candidate-details span { margin-top: 4px; color: #737b91; font-size: 0.74rem; }

.radio-visual {
  display: grid;
  width: 20px;
  height: 20px;
  place-items: center;
  border: 2px solid #aeb5c8;
  border-radius: 50%;
}

.selected .radio-visual { border: 6px solid #4259bb; }

.selection-text {
  position: absolute;
  right: 12px;
  bottom: 6px;
  color: #66708c;
  font-size: 0.62rem;
```

```
}

.vote-actions { margin-top: 22px; }

.vote-button {
  display: flex;
  width: 100%;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 14px 20px;
  color: #fff;
  font-weight: 700;
  cursor: pointer;
  background: #3149a7;
  border: 0;
  border-radius: 12px;
  box-shadow: 0 8px 18px rgba(49, 73, 167, 0.2);
}

.vote-button:hover { background: #263b90; }
.vote-button span { font-size: 1.15rem; }

.feedback {
  min-height: 19px;
  margin: 10px 0 0;
  color: #747d94;
  font-size: 0.78rem;
  text-align: center;
}

.feedback.error { color: #ae3546; font-weight: 600; }
.feedback.success { color: #08786f; font-weight: 600; }

.results-topline {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 14px;
}

.total-card {
  flex: 0 0 auto;
  padding: 10px 14px;
  text-align: right;
  background: #eef9f7;
  border: 1px solid #d4eeea;
  border-radius: 11px;
}

.total-card span {
  display: block;
  color: #5c7775;
  font-size: 0.66rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.total-card strong { display: block; color: #087c73; font-size: 1.35rem; line-height: 1.15; }
.result-list { display: grid; gap: 21px; margin-top: 30px; }
.result-labels, .result-labels > div { display: flex; align-items: center; }

.result-labels {
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 8px;
}

.result-labels > div { min-width: 0; gap: 8px; }
.result-labels strong { font-size: 0.85rem; }
.result-labels span { color: #7a8296; font-size: 0.7rem; white-space: nowrap; }
.result-labels .percentage { color: #3149a7; }

.progress-track {
```

```
  height: 8px;
  overflow: hidden;
  background: #e8ebf2;
  border-radius: 99px;
}

.progress-fill {
  display: block;
  height: 100%;
  max-width: 100%;
  background: linear-gradient(90deg, #324ba9, #2aa79c);
  border-radius: inherit;
  transition: width 300ms ease;
}

.leader-summary {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 13px 15px;
  margin: 27px 0 0;
  color: #2a3a51;
  font-size: 0.8rem;
  font-weight: 700;
  background: #f5f7fa;
  border: 1px solid #e6e9ef;
  border-radius: 11px;
}

.leader-summary span { color: #1ba196; font-size: 0.68rem; }

.page-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  padding: 18px 26px;
  background: #fff;
  border: 1px solid #e3e7f0;
  border-top: 0;
  border-radius: 0 0 20px 20px;
}

.page-footer strong { font-size: 0.78rem; }
.page-footer p { margin: 3px 0 0; color: #777f92; font-size: 0.7rem; }

.reset-button {
  flex: 0 0 auto;
  padding: 9px 13px;
  color: #a33c4c;
  font-size: 0.76rem;
  font-weight: 700;
  cursor: pointer;
  background: #fff5f6;
  border: 1px solid #efccd2;
  border-radius: 9px;
}

.reset-button:hover { background: #feecef; }

@media (max-width: 880px) {
  .workspace { grid-template-columns: 1fr; }
  .voting-panel { border-right: 0; border-bottom: 1px solid #e6e9f1; }
}

@media (max-width: 580px) {
  .page-shell { width: min(100% - 24px, 1120px); padding: 12px 0; }
  .hero { padding: 24px 22px 26px; border-radius: 18px 18px 6px 6px; }
  .brand-row { margin-bottom: 25px; }
  .panel { padding: 25px 20px; }
  .candidate-grid { grid-template-columns: 1fr; }
  .candidate-card { min-height: 82px; }
  .results-topline { align-items: stretch; flex-direction: column; }

  .total-card {
```

```
    display: flex;
    align-items: center;
    justify-content: space-between;
    text-align: left;
  }

  .result-labels > div { align-items: flex-start; flex-direction: column; gap: 2px; }
  .page-footer { align-items: stretch; flex-direction: column; padding: 18px 20px; }
  .reset-button { width: 100%; }
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    scroll-behavior: auto !important;
    transition-duration: 0.01ms !important;
  }
}
```

PRACTICAL 10

10. Develop a travel log App. In which user can view/share travel experiences.

1. server.js

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const travelRoutes = require('./routes/travelRoutes');
dotenv.config();

const app = express();
const PORT = process.env.PORT || 3000;

// Connect to MongoDB.
connectDB();

// Middleware to read JSON and form data.
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve frontend files from the public folder.
app.use(express.static('public'));

// Simple route used to check whether the API is running.
app.get('/api/health', (req, res) => {
  res.json({
    success: true,
    message: 'Travel Log API is running'
  });
});

// Travel CRUD routes.
app.use('/api/travels', travelRoutes);

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

2. config/db.js

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected successfully');
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
  }
};

module.exports = connectDB;
```

3. models/Travel.js

```
const mongoose = require('mongoose');

const travelSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  location: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    required: true,
    trim: true
  },
  travelDate: {
```

```
    type: Date,
    required: true
  },
  imageUrl: {
    type: String,
    default: '',
    trim: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('Travel', travelSchema);
```

4. controllers/travelController.js

```
const mongoose = require('mongoose');
const Travel = require('../models/Travel');

// Create a new travel experience.
const createTravel = async (req, res) => {
  try {
    const travel = await Travel.create({
      title: req.body.title,
      location: req.body.location,
      description: req.body.description,
      travelDate: req.body.travelDate,
      imageUrl: req.body.imageUrl
    });

    res.status(201).json({
      success: true,
      message: 'Travel experience created successfully',
      data: travel
    });
  } catch (error) {
    if (error.name === 'ValidationError' || error.name === 'CastError') {
      return res.status(400).json({
        success: false,
        message: 'Please provide valid travel details'
      });
    }

    res.status(500).json({
      success: false,
      message: 'Unable to create travel experience'
    });
  }
};

// Get all travel experiences, newest first.
const getTravels = async (req, res) => {
  try {
    const travels = await Travel.find().sort({ createdAt: -1 });

    res.status(200).json({
      success: true,
      count: travels.length,
      data: travels
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Unable to fetch travel experiences'
    });
  }
};

// Get one travel experience by its MongoDB ID.
const getTravelById = async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({
        success: false,
```

```
        message: 'Invalid travel ID'
      });
    }

    const travel = await Travel.findById(req.params.id);

    if (!travel) {
      return res.status(404).json({
        success: false,
        message: 'Travel experience not found'
      });
    }

    res.status(200).json({
      success: true,
      data: travel
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Unable to fetch travel experience'
    });
  }
};

// Update an existing travel experience.
const updateTravel = async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({
        success: false,
        message: 'Invalid travel ID'
      });
    }

    const allowedFields = [
      'title',
      'location',
      'description',
      'travelDate',
      'imageUrl'
    ];
    const updates = {};

    allowedFields.forEach((field) => {
      if (req.body[field] !== undefined) {
        updates[field] = req.body[field];
      }
    });

    const travel = await Travel.findByIdAndUpdate(req.params.id, updates, {
      new: true,
      runValidators: true
    });

    if (!travel) {
      return res.status(404).json({
        success: false,
        message: 'Travel experience not found'
      });
    }

    res.status(200).json({
      success: true,
      message: 'Travel experience updated successfully',
      data: travel
    });
  } catch (error) {
    if (error.name === 'ValidationError' || error.name === 'CastError') {
      return res.status(400).json({
        success: false,
        message: 'Please provide valid travel details'
      });
    }
    res.status(500).json({
```

```
      success: false,
      message: 'Unable to update travel experience'
    });
  }
};

// Delete a travel experience.
const deleteTravel = async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({
        success: false,
        message: 'Invalid travel ID'
      });
    }

    const travel = await Travel.findByIdAndDelete(req.params.id);

    if (!travel) {
      return res.status(404).json({
        success: false,
        message: 'Travel experience not found'
      });
    }

    res.status(200).json({
      success: true,
      message: 'Travel experience deleted successfully'
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Unable to delete travel experience'
    });
  }
};

module.exports = {
  createTravel,
  getTravels,
  getTravelById,
  updateTravel,
  deleteTravel
};
```

5. routes/travelRoutes.js

```
const express = require('express');
const {
  createTravel,
  getTravels,
  getTravelById,
  updateTravel,
  deleteTravel
} = require('../controllers/travelController');

const router = express.Router();

router.post('/', createTravel);
router.get('/', getTravels);
router.get('/:id', getTravelById);
router.put('/:id', updateTravel);
router.delete('/:id', deleteTravel);

module.exports = router;
```

6. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Explore and share memorable travel experiences.">
  <title>Travel Log</title>
  <link rel="stylesheet" href="css/style.css">
```

```
</head>
<body>
  <header class="site-header">
    <div class="header-content">
      <div>
        <a class="site-title" href="index.html">Travel Log</a>
        <p class="site-subtitle">Explore and share travel experiences</p>
      </div>

      <nav class="main-nav" aria-label="Main navigation">
        <a class="active" href="index.html">Home</a>
        <a href="add-travel.html">Share Experience</a>
      </nav>
    </div>
  </header>

  <main class="page-container">
    <section class="section-heading">
      <div>
        <p class="eyebrow">Shared journeys</p>
        <h1>Travel Experiences</h1>
      </div>
      <a class="button button-primary" href="add-travel.html">Share Your Experience</a>
    </section>

    <div id="feedback" class="feedback" role="status" aria-live="polite"></div>

    <div id="travel-list" class="travel-grid" aria-live="polite">
      <p class="loading-message">Loading travel experiences...</p>
    </div>
  </main>

  <footer class="site-footer">
    <p>Travel Log &amp;mdash; Keep your journeys remembered.</p>
  </footer>

  <script src="js/app.js"></script>
</body>
</html>
```

7. public/add-travel.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Share a memorable travel experience.">
  <title>Share Travel Experience | Travel Log</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header class="site-header">
    <div class="header-content">
      <div>
        <a class="site-title" href="index.html">Travel Log</a>
        <p class="site-subtitle">Explore and share travel experiences</p>
      </div>

      <nav class="main-nav" aria-label="Main navigation">
        <a href="index.html">Home</a>
        <a class="active" href="add-travel.html">Share Experience</a>
      </nav>
    </div>
  </header>

  <main class="page-container form-page-container">
    <section class="form-introduction">
      <p class="eyebrow">Create a travel log</p>
      <h1>Share Travel Experience</h1>
      <p>Share your memorable journey with others.</p>
    </section>

    <form id="travel-form" class="travel-form">
      <div id="form-message" class="form-message" role="status" aria-live="polite"></div>
      <div class="form-group">
```

```
      <label for="title">Travel Title <span aria-hidden="true">*</span></label>
      <input id="title" name="title" type="text" placeholder="Trip to Goa" required>
    </div>

    <div class="form-group">
      <label for="location">Location <span aria-hidden="true">*</span></label>
      <input id="location" name="location" type="text" placeholder="Goa, India" required>
    </div>

    <div class="form-group">
      <label for="travelDate">Travel Date <span aria-hidden="true">*</span></label>
      <input id="travelDate" name="travelDate" type="date" required>
    </div>

    <div class="form-group">
      <label for="imageUrl">Image URL</label>
      <input
        id="imageUrl"
        name="imageUrl"
        type="url"
        placeholder="https://example.com/travel.jpg"
        aria-describedby="image-help"
      >
      <small id="image-help">Optional - leave blank to use the default travel image.</small>
    </div>

    <div class="form-group">
      <label for="description">Travel Experience <span aria-hidden="true">*</span></label>
      <textarea
        id="description"
        name="description"
        rows="7"
        placeholder="Share your travel experience..."
        required
      ></textarea>
    </div>

    <div class="form-actions">
      <button id="submit-button" class="button button-primary" type="submit">
        Share Experience
      </button>
      <a class="button button-secondary" href="index.html">Cancel</a>
    </div>
  </form>
 </main>

 <footer class="site-footer">
  <p>Travel Log &mdash; Keep your journeys remembered.</p>
 </footer>

 <script src="js/add-travel.js"></script>
</body>
</html>
```

8. public/edit-travel.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <meta name="description" content="Edit a saved travel experience.">
 <title>Edit Travel Experience | Travel Log</title>
 <link rel="stylesheet" href="css/style.css">
</head>
<body>
 <header class="site-header">
  <div class="header-content">
   <div>
    <a class="site-title" href="index.html">Travel Log</a>
    <p class="site-subtitle">Explore and share travel experiences</p>
   </div>

   <nav class="main-nav" aria-label="Main navigation">
    <a href="index.html">Home</a>
    <a href="add-travel.html">Share Experience</a>
```

```
    </nav>
   </div>
  </header>

  <main class="page-container form-page-container">
   <section class="form-introduction">
    <p class="eyebrow">Update your journey</p>
    <h1>Edit Travel Experience</h1>
    <p>Make changes to your saved travel experience.</p>
   </section>

   <div
    id="form-message"
    class="form-message show info"
    role="status"
    aria-live="polite"
   >Loading travel experience...</div>

   <form id="edit-travel-form" class="travel-form" hidden>
    <div class="form-group">
     <label for="title">Travel Title <span aria-hidden="true">*</span></label>
     <input id="title" name="title" type="text" required>
    </div>

    <div class="form-group">
     <label for="location">Location <span aria-hidden="true">*</span></label>
     <input id="location" name="location" type="text" required>
    </div>

    <div class="form-group">
     <label for="travelDate">Travel Date <span aria-hidden="true">*</span></label>
     <input id="travelDate" name="travelDate" type="date" required>
    </div>

    <div class="form-group">
     <label for="imageUrl">Image URL</label>
     <input
      id="imageUrl"
      name="imageUrl"
      type="url"
      placeholder="https://example.com/travel.jpg"
      aria-describedby="image-help"
     >
     <small id="image-help">Optional - leave blank to use the default travel image.</small>
    </div>

    <div class="form-group">
     <label for="description">Travel Experience <span aria-hidden="true">*</span></label>
     <textarea id="description" name="description" rows="7" required></textarea>
    </div>

    <div class="form-actions">
     <button id="update-button" class="button button-primary" type="submit" disabled>
      Update Experience
     </button>
     <a class="button button-secondary" href="index.html">Cancel</a>
    </div>
   </form>
  </main>

  <footer class="site-footer">
   <p>Travel Log &amp;mdash; Keep your journeys remembered.</p>
  </footer>

  <script src="js/edit-travel.js"></script>
 </body>
</html>
```

9. public/css/style.css

```
:root {
  --primary: #087f5b;
  --primary-dark: #056847;
  --danger: #c92a2a;
  --danger-dark: #a51111;
  --ink: #18312b;
```

```
  --muted: #60736e;
  --surface: #ffffff;
  --background: #f2f7f5;
  --border: #dce8e3;
  --shadow: 0 10px 28px rgba(24, 49, 43, 0.1);
}

* {
  box-sizing: border-box;
}

body {
  min-height: 100vh;
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  color: var(--ink);
  background: var(--background);
  line-height: 1.6;
}

a {
  color: inherit;
}

.site-header {
  color: #fff;
  background: linear-gradient(120deg, #075f46, #0ca678);
  box-shadow: 0 4px 16px rgba(5, 104, 71, 0.22);
}

.header-content,
.page-container {
  width: min(1120px, calc(100% - 40px));
  margin: 0 auto;
}

.header-content {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 32px;
  padding: 24px 0;
}

.site-title {
  display: inline-block;
  color: #fff;
  font-size: 2rem;
  font-weight: 800;
  line-height: 1.1;
  text-decoration: none;
}

.site-subtitle {
  margin: 6px 0 0;
  color: #d7f5eb;
}

.main-nav {
  display: flex;
  align-items: center;
  gap: 10px;
}

.main-nav a {
  padding: 9px 14px;
  border-radius: 7px;
  color: #fff;
  font-weight: 700;
  text-decoration: none;
}

.main-nav a:hover,
.main-nav a:focus-visible,
.main-nav a.active {
```

```
  background-color: rgba(255, 255, 255, 0.17);
}

.page-container {
  min-height: calc(100vh - 230px);
  padding: 54px 0 72px;
}

.section-heading {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 24px;
  margin-bottom: 30px;
}

.section-heading h1 {
  margin: 2px 0 0;
  font-size: clamp(1.8rem, 4vw, 2.6rem);
  line-height: 1.2;
}

.eyebrow {
  margin: 0;
  color: var(--primary);
  font-size: 0.82rem;
  font-weight: 800;
  letter-spacing: 0.12em;
  text-transform: uppercase;
}

.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  min-height: 42px;
  padding: 10px 17px;
  border: 0;
  border-radius: 7px;
  font: inherit;
  font-weight: 700;
  text-decoration: none;
  cursor: pointer;
  transition: background-color 0.2s, transform 0.2s;
}

.button:hover {
  transform: translateY(-1px);
}

.button-primary {
  color: #fff;
  background-color: var(--primary);
}

.button-primary:hover,
.button-primary:focus-visible {
  background-color: var(--primary-dark);
}

.button-secondary {
  color: var(--primary-dark);
  background-color: #e6f5ef;
}

.button-secondary:hover,
.button-secondary:focus-visible {
  background-color: #ccecdf;
}

.button-danger {
  color: #fff;
  background-color: var(--danger);
}
.button-danger:hover,
```

```
.button-danger:focus-visible {
  background-color: var(--danger-dark);
}

.button:disabled {
  cursor: not-allowed;
  opacity: 0.65;
  transform: none;
}

.feedback {
  display: none;
  margin-bottom: 22px;
  padding: 12px 16px;
  border-radius: 7px;
  font-weight: 700;
}

.feedback.show {
  display: block;
}

.feedback.success {
  color: #135d42;
  background-color: #dff6ec;
  border: 1px solid #afe0cc;
}

.feedback.error {
  color: #8d1717;
  background-color: #fff0f0;
  border: 1px solid #f3bcbc;
}

.travel-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 26px;
}

.travel-card {
  display: flex;
  min-width: 0;
  flex-direction: column;
  overflow: hidden;
  border: 1px solid var(--border);
  border-radius: 14px;
  background-color: var(--surface);
  box-shadow: var(--shadow);
}

.travel-image-wrapper {
  height: 210px;
  overflow: hidden;
  background: linear-gradient(135deg, #c8eee1, #eaf8f3);
}

.travel-image {
  width: 100%;
  height: 100%;
  display: block;
  object-fit: cover;
}

.image-placeholder {
  display: flex;
  height: 100%;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 6px;
  color: var(--primary-dark);
  font-weight: 700;
  text-align: center;
}
```

```
.placeholder-icon {
  font-size: 2.4rem;
  line-height: 1;
}

.travel-card-content {
  display: flex;
  flex: 1;
  flex-direction: column;
  padding: 22px;
}

.travel-card h2 {
  margin: 0 0 10px;
  font-size: 1.35rem;
  line-height: 1.3;
}

.travel-meta {
  margin: 3px 0;
  color: var(--muted);
  font-size: 0.94rem;
}

.travel-description {
  flex: 1;
  margin: 16px 0 22px;
  overflow-wrap: anywhere;
}

.card-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

.card-actions .button {
  flex: 1;
}

.loading-message,
.empty-state,
.error-state {
  grid-column: 1 / -1;
  padding: 52px 24px;
  border: 1px dashed #b7ccc4;
  border-radius: 14px;
  background-color: var(--surface);
  color: var(--muted);
  text-align: center;
}

.empty-state h2,
.error-state h2 {
  margin: 0 0 6px;
  color: var(--ink);
}

.empty-state p,
.error-state p {
  margin: 0 0 20px;
}

.site-footer {
  padding: 22px 20px;
  color: var(--muted);
  background-color: #e4efeb;
  text-align: center;
}

.site-footer p {
  margin: 0;
}

/* Add Travel page */
```

```
.form-page-container {
  width: min(760px, calc(100% - 40px));
}

.form-introduction {
  margin-bottom: 26px;
  text-align: center;
}

.form-introduction h1 {
  margin: 3px 0 6px;
  font-size: clamp(1.8rem, 4vw, 2.5rem);
  line-height: 1.2;
}

.form-introduction > p:last-child {
  margin: 0;
  color: var(--muted);
}

.travel-form {
  padding: clamp(24px, 5vw, 40px);
  border: 1px solid var(--border);
  border-radius: 14px;
  background-color: var(--surface);
  box-shadow: var(--shadow);
}

.form-group {
  margin-bottom: 22px;
}

.form-group label {
  display: block;
  margin-bottom: 7px;
  font-weight: 700;
}

.form-group label span {
  color: var(--danger);
}

.form-group input,
.form-group textarea {
  width: 100%;
  padding: 12px 14px;
  border: 1px solid #b9cbc4;
  border-radius: 7px;
  color: var(--ink);
  background-color: #fff;
  font: inherit;
}

.form-group textarea {
  min-height: 150px;
  resize: vertical;
}

.form-group input:focus,
.form-group textarea:focus {
  border-color: var(--primary);
  outline: 3px solid rgba(8, 127, 91, 0.14);
}

.form-group small {
  display: block;
  margin-top: 6px;
  color: var(--muted);
}

.form-message {
  display: none;
  margin-bottom: 22px;
  padding: 12px 15px;
  border-radius: 7px;
```

```
  font-weight: 700;
}

.form-message.show {
  display: block;
}

.form-message.success {
  color: #135d42;
  background-color: #dff6ec;
  border: 1px solid #afe0cc;
}

.form-message.error {
  color: #8d1717;
  background-color: #fff0f0;
  border: 1px solid #f3bcbc;
}

.form-message.info {
  color: #25594a;
  background-color: #e8f5f0;
  border: 1px solid #bedfd2;
}

.form-actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 8px;
}

.form-actions .button {
  min-width: 150px;
}

@media (max-width: 680px) {
  .header-content,
  .section-heading {
    align-items: stretch;
    flex-direction: column;
  }

  .header-content {
    gap: 18px;
  }

  .main-nav {
    align-items: stretch;
  }

  .main-nav a {
    flex: 1;
    text-align: center;
  }

  .page-container {
    padding-top: 38px;
  }

  .section-heading .button {
    align-self: flex-start;
  }
}

@media (max-width: 420px) {
  .header-content,
  .page-container {
    width: min(100% - 24px, 1120px);
  }

  .main-nav,
  .card-actions {
    flex-direction: column;
  }
```

```
  .section-heading .button {
    width: 100%;
  }

  .travel-grid {
    grid-template-columns: 1fr;
  }

  .form-page-container {
    width: min(100% - 24px, 760px);
  }

  .form-actions {
    flex-direction: column;
  }

  .form-actions .button {
    width: 100%;
  }
}
```

10. public/js/app.js

```
const travelList = document.getElementById('travel-list');
const feedback = document.getElementById('feedback');

// Convert the MongoDB date into a simple readable date.
const formatDate = (dateValue) => {
  const date = new Date(dateValue);

  if (Number.isNaN(date.getTime())) {
    return 'Date unavailable';
  }

  return date.toLocaleDateString('en-IN', {
    day: 'numeric',
    month: 'short',
    year: 'numeric',
    timeZone: 'UTC'
  });
};

const createTextElement = (tagName, className, text) => {
  const element = document.createElement(tagName);
  element.className = className;
  element.textContent = text;
  return element;
};

const addImagePlaceholder = (container) => {
  container.replaceChildren();

  const placeholder = document.createElement('div');
  placeholder.className = 'image-placeholder';

  const icon = createTextElement('span', 'placeholder-icon', '✈');
  icon.setAttribute('aria-hidden', 'true');

  placeholder.append(icon);
  placeholder.append(createTextElement('span', '', 'Travel image unavailable'));
  container.append(placeholder);
};

const addTravelImage = (container, travel) => {
  if (!travel.imageUrl) {
    addImagePlaceholder(container);
    return;
  }

  try {
    const imageUrl = new URL(travel.imageUrl, window.location.origin);

    if (!['http:', 'https:'].includes(imageUrl.protocol)) {
      addImagePlaceholder(container);
      return;
    }
```

```
    const image = document.createElement('img');
    image.className = 'travel-image';
    image.src = imageUrl.href;
    image.alt = `Travel experience: ${travel.title}`;
    image.addEventListener('error', () => addImagePlaceholder(container));
    container.append(image);
  } catch (error) {
    addImagePlaceholder(container);
  }
};

const showFeedback = (message, type) => {
  feedback.textContent = message;
  feedback.className = `feedback show ${type}`;
};

const showEmptyState = () => {
  travelList.replaceChildren();

  const emptyState = document.createElement('div');
  emptyState.className = 'empty-state';
  emptyState.append(createTextElement('h2', '', 'No travel experiences yet.'));
  emptyState.append(createTextElement('p', '', 'Be the first to share your journey!'));

  const shareLink = createTextElement('a', 'button button-primary', 'Share Experience');
  shareLink.href = 'add-travel.html';
  emptyState.append(shareLink);
  travelList.append(emptyState);
};

const showLoadError = () => {
  travelList.replaceChildren();

  const errorState = document.createElement('div');
  errorState.className = 'error-state';
  errorState.append(createTextElement('h2', '', 'Unable to load travel experiences.'));
  errorState.append(createTextElement('p', '', 'Please try again.'));
  travelList.append(errorState);
};

const deleteTravel = async (travelId, deleteButton) => {
  const shouldDelete = confirm(
    'Are you sure you want to delete this travel experience?'
  );

  if (!shouldDelete) {
    return;
  }

  deleteButton.disabled = true;
  deleteButton.textContent = 'Deleting...';

  try {
    const response = await fetch(`/api/travels/${travelId}`, {
      method: 'DELETE'
    });

    const result = await response.json();

    if (!response.ok || !result.success) {
      throw new Error('Delete request failed');
    }

    await loadTravels();
    showFeedback('Travel experience deleted successfully.', 'success');
  } catch (error) {
    deleteButton.disabled = false;
    deleteButton.textContent = 'Delete';
    showFeedback('Unable to delete travel experience. Please try again.', 'error');
  }
};

const createTravelCard = (travel) => {
  const card = document.createElement('article');
  card.className = 'travel-card';
```

```
  const imageWrapper = document.createElement('div');
  imageWrapper.className = 'travel-image-wrapper';
  addTravelImage(imageWrapper, travel);

  const content = document.createElement('div');
  content.className = 'travel-card-content';
  content.append(createTextElement('h2', '', travel.title));
  content.append(createTextElement('p', 'travel-meta', `Location: ${travel.location}`));
  content.append(createTextElement('p', 'travel-meta', `Date: ${formatDate(travel.travelDate)}`));
  content.append(createTextElement('p', 'travel-description', travel.description));

  const actions = document.createElement('div');
  actions.className = 'card-actions';

  const editLink = createTextElement('a', 'button button-secondary', 'Edit');
  editLink.href = `edit-travel.html?id=${encodeURIComponent(travel._id)}`;

  const deleteButton = createTextElement('button', 'button button-danger', 'Delete');
  deleteButton.type = 'button';
  deleteButton.addEventListener('click', () => deleteTravel(travel._id, deleteButton));

  actions.append(editLink, deleteButton);
  content.append(actions);
  card.append(imageWrapper, content);

  return card;
};

// Fetch and display every travel experience from the backend API.
const loadTravels = async () => {
  travelList.replaceChildren(
    createTextElement('p', 'loading-message', 'Loading travel experiences...')
  );

  try {
    const response = await fetch('/api/travels');
    const result = await response.json();

    if (!response.ok || !result.success) {
      throw new Error('Travel request failed');
    }

    if (result.data.length === 0) {
      showEmptyState();
      return;
    }

    travelList.replaceChildren();
    result.data.forEach((travel) => {
      travelList.append(createTravelCard(travel));
    });
  } catch (error) {
    showLoadError();
  }
};

document.addEventListener('DOMContentLoaded', loadTravels);
```

11. public/js/add-travel.js

```
const travelForm = document.getElementById('travel-form');
const formMessage = document.getElementById('form-message');
const submitButton = document.getElementById('submit-button');

const showFormMessage = (message, type) => {
  formMessage.textContent = message;
  formMessage.className = `form-message show ${type}`;
};

// Show a clear message when native HTML validation finds an invalid field.
travelForm.addEventListener(
  'invalid',
  (event) => {
    if (event.target.validity.typeMismatch) {
      showFormMessage('Please enter a valid image URL.', 'error');
    } else {
```

```
      showFormMessage('Please fill in all required fields.', 'error');
    }
  },
  true
);

travelForm.addEventListener('submit', async (event) => {
  event.preventDefault();

  const title = document.getElementById('title').value.trim();
  const location = document.getElementById('location').value.trim();
  const travelDate = document.getElementById('travelDate').value;
  const imageUrl = document.getElementById('imageUrl').value.trim();
  const description = document.getElementById('description').value.trim();

  if (!title || !location || !travelDate || !description) {
    showFormMessage('Please fill in all required fields.', 'error');
    return;
  }

  submitButton.disabled = true;
  submitButton.textContent = 'Sharing...';
  formMessage.className = 'form-message';
  formMessage.textContent = '';

  try {
    const response = await fetch('/api/travels', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title,
        location,
        description,
        travelDate,
        imageUrl
      })
    });

    const result = await response.json();

    if (!response.ok || !result.success) {
      throw new Error(result.message || 'Unable to share travel experience. Please try again.');
    }

    showFormMessage('Travel experience shared successfully!', 'success');
    travelForm.reset();

    setTimeout(() => {
      window.location.href = 'index.html';
    }, 800);
  } catch (error) {
    showFormMessage(
      error.message || 'Unable to share travel experience. Please try again.',
      'error'
    );
    submitButton.disabled = false;
    submitButton.textContent = 'Share Experience';
  }
});
```

12. public/js/edit-travel.js

```
const editForm = document.getElementById('edit-travel-form');
const formMessage = document.getElementById('form-message');
const updateButton = document.getElementById('update-button');

const params = new URLSearchParams(window.location.search);
const travelId = params.get('id');

const showFormMessage = (message, type) => {
  formMessage.textContent = message;
  formMessage.className = `form-message show ${type}`;
};
const hideFormMessage = () => {
```

```
  formMessage.textContent = '';
  formMessage.className = 'form-message';
};

const loadTravel = async () => {
  if (!travelId) {
    showFormMessage('Invalid travel experience.', 'error');
    return;
  }

  try {
    const response = await fetch(`/api/travels/${travelId}`);
    const result = await response.json();

    if (response.status === 400) {
      showFormMessage('Invalid travel experience.', 'error');
      return;
    }

    if (response.status === 404) {
      showFormMessage('Travel experience not found.', 'error');
      return;
    }

    if (!response.ok || !result.success) {
      throw new Error('Unable to load travel experience');
    }

    document.getElementById('title').value = result.data.title;
    document.getElementById('location').value = result.data.location;
    document.getElementById('travelDate').value = result.data.travelDate.split('T')[0];
    document.getElementById('imageUrl').value = result.data.imageUrl || '';
    document.getElementById('description').value = result.data.description;

    editForm.hidden = false;
    updateButton.disabled = false;
    hideFormMessage();
  } catch (error) {
    showFormMessage('Unable to load travel experience. Please try again.', 'error');
  }
};

// Show a clear message when native HTML validation finds an invalid field.
editForm.addEventListener(
  'invalid',
  (event) => {
    if (event.target.validity.typeMismatch) {
      showFormMessage('Please enter a valid image URL.', 'error');
    } else {
      showFormMessage('Please fill in all required fields.', 'error');
    }
  },
  true
);

editForm.addEventListener('submit', async (event) => {
  event.preventDefault();

  const title = document.getElementById('title').value.trim();
  const location = document.getElementById('location').value.trim();
  const travelDate = document.getElementById('travelDate').value;
  const imageUrl = document.getElementById('imageUrl').value.trim();
  const description = document.getElementById('description').value.trim();

  if (!title || !location || !travelDate || !description) {
    showFormMessage('Please fill in all required fields.', 'error');
    return;
  }

  updateButton.disabled = true;
  updateButton.textContent = 'Updating...';
  hideFormMessage();

  try {
    const response = await fetch(`/api/travels/${travelId}`, {
```

```
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        title,
        location,
        description,
        travelDate,
        imageUrl
      })
    });

    const result = await response.json();

    if (!response.ok || !result.success) {
      throw new Error(result.message || 'Unable to update travel experience. Please try again.');
    }

    showFormMessage('Travel experience updated successfully!', 'success');

    setTimeout(() => {
      window.location.href = 'index.html';
    }, 800);
  } catch (error) {
    showFormMessage(
      error.message || 'Unable to update travel experience. Please try again.',
      'error'
    );
    updateButton.disabled = false;
    updateButton.textContent = 'Update Experience';
  }
});

document.addEventListener('DOMContentLoaded', loadTravel);
```

PRACTICAL 11

11. Develop an app that recommends the food as per nutrition requirements.

1. server.js

```
const express = require("express");
const path = require("path");
const foods = require("./foods");

const app = express();
const PORT = process.env.PORT || 3000;
const supportedGoals = ["High Protein", "Low Calorie", "Balanced Diet"];

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/api/foods", (req, res) => {
  res.json({ success: true, count: foods.length, foods });
});

app.post("/api/recommend", (req, res) => {
  const body = req.body || {};
  const { goal, maxCalories, minProtein } = body;

  if (typeof goal !== "string" || goal.trim() === "") {
    return res.status(400).json({
      success: false,
      message: "Please select a nutrition goal."
    });
  }

  if (!supportedGoals.includes(goal)) {
    return res.status(400).json({
      success: false,
      message: "Please select a supported nutrition goal."
    });
  }

  if (maxCalories === undefined || maxCalories === null || maxCalories === "") {
    return res.status(400).json({
      success: false,
      message: "Please enter a calorie limit."
    });
  }

  const calorieLimit = Number(maxCalories);
  if (!Number.isFinite(calorieLimit) || calorieLimit <= 0) {
    return res.status(400).json({
      success: false,
      message: "Please enter a valid calorie limit."
    });
  }

  if (minProtein === undefined || minProtein === null || minProtein === "") {
    return res.status(400).json({
      success: false,
      message: "Please enter a minimum protein value."
    });
  }

  const proteinMinimum = Number(minProtein);
  if (!Number.isFinite(proteinMinimum) || proteinMinimum < 0) {
    return res.status(400).json({
      success: false,
      message: "Please enter a valid minimum protein value."
    });
  }

  const recommendations = foods
    .filter((food) =>
      food.goals.includes(goal) &&
      food.calories <= calorieLimit &&
      food.protein >= proteinMinimum
    )
    .sort((first, second) =>
```

```
      second.protein - first.protein || first.calories - second.calories
    );

  const response = {
    success: true,
    count: recommendations.length,
    foods: recommendations
  };

  if (recommendations.length === 0) {
    response.message = "No exact food match found for these nutrition requirements.";
  }

  return res.status(200).json(response);
});

app.use((error, req, res, next) => {
  if (error instanceof SyntaxError && error.status === 400 && "body" in error) {
    return res.status(400).json({
      success: false,
      message: "The request contains invalid JSON."
    });
  }

  console.error("Unexpected server error:", error.message);
  return res.status(500).json({
    success: false,
    message: "Unable to process the request right now."
  });
});

app.listen(PORT, () => {
  console.log(`NutriPick is running at http://localhost:${PORT}`);
});
```

2. foods.js

```
// Approximate nutrition values per serving for this educational demo.
const foods = [
  {
    id: 1,
    name: "Grilled Chicken Breast",
    category: "Protein",
    calories: 250,
    protein: 40,
    carbs: 0,
    fat: 7,
    goals: ["High Protein", "Balanced Diet"],
    description: "A lean and filling protein-rich meal option."
  },
  {
    id: 2,
    name: "Boiled Eggs",
    category: "Protein",
    calories: 156,
    protein: 13,
    carbs: 1,
    fat: 11,
    goals: ["High Protein", "Low Calorie"],
    description: "A simple source of protein for a light meal."
  },
  {
    id: 3,
    name: "Greek Yogurt",
    category: "Dairy",
    calories: 120,
    protein: 15,
    carbs: 8,
    fat: 3,
    goals: ["High Protein", "Low Calorie", "Balanced Diet"],
    description: "Creamy yogurt with protein and moderate calories."
  },
  {
    id: 4,
    name: "Paneer",
    category: "Dairy",
```

```
  calories: 300,
  protein: 20,
  carbs: 6,
  fat: 22,
  goals: ["High Protein", "Balanced Diet"],
  description: "A vegetarian protein option made from milk."
},
{
  id: 5,
  name: "Dal",
  category: "Legumes",
  calories: 230,
  protein: 14,
  carbs: 36,
  fat: 4,
  goals: ["Low Calorie", "Balanced Diet"],
  description: "Cooked lentils offering protein, fibre and carbohydrates."
},
{
  id: 6,
  name: "Chickpea Salad",
  category: "Salad",
  calories: 280,
  protein: 12,
  carbs: 42,
  fat: 8,
  goals: ["Balanced Diet"],
  description: "Chickpeas and fresh vegetables in a balanced bowl."
},
{
  id: 7,
  name: "Oatmeal",
  category: "Breakfast",
  calories: 210,
  protein: 8,
  carbs: 36,
  fat: 4,
  goals: ["Low Calorie", "Balanced Diet"],
  description: "A fibre-rich breakfast with steady carbohydrates."
},
{
  id: 8,
  name: "Vegetable Bowl",
  category: "Vegetables",
  calories: 160,
  protein: 6,
  carbs: 28,
  fat: 4,
  goals: ["Low Calorie", "Balanced Diet"],
  description: "A colourful bowl of lightly cooked seasonal vegetables."
},
{
  id: 9,
  name: "Brown Rice Bowl",
  category: "Grains",
  calories: 340,
  protein: 9,
  carbs: 68,
  fat: 4,
  goals: ["Balanced Diet"],
  description: "Whole-grain rice served with mixed vegetables."
},
{
  id: 10,
  name: "Tofu",
  category: "Protein",
  calories: 180,
  protein: 20,
  carbs: 5,
  fat: 10,
  goals: ["High Protein", "Low Calorie", "Balanced Diet"],
  description: "A versatile plant-based protein with mild flavour."
},
{
  id: 11,
```

```
    name: "Fruit Yogurt Bowl",
    category: "Breakfast",
    calories: 240,
    protein: 10,
    carbs: 38,
    fat: 6,
    goals: ["Low Calorie", "Balanced Diet"],
    description: "Yogurt and fresh fruit for a refreshing balanced snack."
  },
  {
    id: 12,
    name: "Sprout Salad",
    category: "Salad",
    calories: 140,
    protein: 9,
    carbs: 24,
    fat: 2,
    goals: ["Low Calorie", "Balanced Diet"],
    description: "Crunchy sprouts and vegetables with light seasoning."
  }
];

module.exports = foods;
```

3. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="NutriPick recommends foods using simple nutrition requirements.">
  <title>NutriPick | Smart Food Choices</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <a class="brand" href="#top" aria-label="NutriPick home">
      <span class="brand-mark" aria-hidden="true">N</span>
      <span>NutriPick</span>
    </a>
    <a class="header-link" href="#recommendation">Find My Foods</a>
  </header>

  <main id="top">
    <section class="hero" aria-labelledby="hero-title">
      <div class="hero-copy">
        <p class="eyebrow">Simple nutrition. Smarter choices.</p>
        <h1 id="hero-title">Find Food That Fits Your Nutrition Goals.</h1>
        <p class="hero-text">
          Enter your nutrition requirements and discover suitable food options.
        </p>
        <a class="primary-link" href="#recommendation">Find My Foods <span aria-hidden="true">→</span></a>
      </div>

      <div class="hero-visual" aria-hidden="true">
        <div class="plate">
          <span class="leaf leaf-one"></span>
          <span class="leaf leaf-two"></span>
          <span class="food-shape tomato"></span>
          <span class="food-shape grain"></span>
          <span class="food-shape greens"></span>
          <span class="food-shape protein"></span>
        </div>
        <span class="mini-card mini-protein">Protein<br><strong>20g+</strong></span>
        <span class="mini-card mini-fresh">Fresh<br><strong>& balanced</strong></span>
      </div>
    </section>

    <section class="workspace" id="recommendation" aria-labelledby="form-title">
      <div class="form-panel">
        <p class="section-label">Personalise your picks</p>
        <h2 id="form-title">Your Nutrition Requirements</h2>
        <p class="panel-intro">Set three simple preferences to find matching foods.</p>

        <form id="nutrition-form" novalidate>
```

```
        <div class="field">
          <label for="goal">Nutrition Goal</label>
          <select id="goal" name="goal" required>
            <option value="">Select a goal</option>
            <option value="High Protein">High Protein</option>
            <option value="Low Calorie">Low Calorie</option>
            <option value="Balanced Diet">Balanced Diet</option>
          </select>
        </div>

        <div class="input-row">
          <div class="field">
            <label for="max-calories">Maximum Calories</label>
            <div class="input-with-unit">
              <input id="max-calories" name="maxCalories" type="number" min="1" step="1" placeholder="400" required>
              <span>kcal</span>
            </div>
          </div>

          <div class="field">
            <label for="min-protein">Minimum Protein</label>
            <div class="input-with-unit">
              <input id="min-protein" name="minProtein" type="number" min="0" step="1" placeholder="15" required>
              <span>g</span>
            </div>
          </div>
        </div>

        <p class="form-error" id="form-error" role="alert"></p>
        <button id="submit-button" type="submit">Recommend Foods</button>
      </form>
    </div>

    <div class="results-panel" aria-labelledby="results-title">
      <div class="results-heading">
        <div>
          <p class="section-label">Your matches</p>
          <h2 id="results-title">Recommended Foods</h2>
        </div>
        <span class="result-count" id="result-count" hidden></span>
      </div>

      <div id="results" class="results" aria-live="polite" aria-busy="false">
        <div class="state-card initial-state">
          <span class="state-icon" aria-hidden="true">✓</span>
          <h3>Your food recommendations will appear here.</h3>
          <p>Enter your nutrition requirements to get started.</p>
        </div>
      </div>
    </div>
  </section>
</main>

<footer>
  <p><strong>NutriPick</strong> — Smart food choices for your nutrition goals.</p>
  <p>Nutrition values are approximate and this app is for educational purposes.</p>
</footer>

<script src="app.js"></script>
</body>
</html>
```

4. public/style.css

```
:root {
  --green-900: #173f35; --green-700: #26735b; --green-500: #45a579;
  --green-100: #dff1e7; --sage-50: #f3f7f2; --cream: #fbfaf5;
  --white: #fff; --charcoal: #21302b; --muted: #66736e;
  --border: #dce5df; --error: #a23b3b;
  --shadow: 0 20px 50px rgba(30, 70, 55, 0.09);
}

* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body {
  margin: 0; min-width: 320px; color: var(--charcoal); background: var(--cream);
```

```
  font-family: "Segoe UI", Arial, sans-serif; line-height: 1.6;
}
a { color: inherit; }

.site-header {
  width: min(1160px, calc(100% - 40px)); margin: 0 auto; padding: 24px 0;
  display: flex; align-items: center; justify-content: space-between;
}
.brand {
  display: inline-flex; align-items: center; gap: 10px; color: var(--green-900);
  font-size: 1.25rem; font-weight: 800; text-decoration: none;
}
.brand-mark {
  width: 38px; height: 38px; display: grid; place-items: center; color: white;
  border-radius: 12px 12px 12px 4px; background: var(--green-700);
}
.header-link { color: var(--green-700); font-size: .93rem; font-weight: 700; text-decoration: none; }

.hero {
  width: min(1160px, calc(100% - 40px)); min-height: 470px; margin: 8px auto 64px;
  padding: 62px 68px; display: grid; grid-template-columns: 1.1fr .9fr;
  align-items: center; gap: 54px; overflow: hidden; border: 1px solid #e4eadd;
  border-radius: 32px; background: var(--sage-50);
}
.eyebrow, .section-label {
  margin: 0 0 10px; color: var(--green-700); font-size: .75rem;
  font-weight: 800; letter-spacing: .13em; text-transform: uppercase;
}
h1, h2, h3, p { margin-top: 0; }
h1 {
  max-width: 650px; margin-bottom: 18px; color: var(--green-900);
  font: clamp(2.7rem, 5vw, 4.7rem)/1.04 Georgia, "Times New Roman", serif;
  letter-spacing: -.04em;
}
.hero-text { max-width: 550px; margin-bottom: 30px; color: var(--muted); font-size: 1.08rem; }
.primary-link {
  display: inline-flex; align-items: center; gap: 14px; padding: 13px 20px;
  color: white; border-radius: 12px; background: var(--green-700);
  font-weight: 750; text-decoration: none;
}
.primary-link:hover, button:hover { background: var(--green-900); }

.hero-visual { min-height: 330px; position: relative; display: grid; place-items: center; }
.plate {
  width: 290px; height: 290px; position: relative; border: 23px solid white;
  border-radius: 50%; background: #e8eadc; box-shadow: var(--shadow);
}
.food-shape, .leaf { position: absolute; display: block; }
.tomato {
  width: 95px; height: 95px; top: 20px; left: 20px; border-radius: 50%;
  background: #e9785b; box-shadow: inset -10px -8px 0 rgba(120, 50, 30, .11);
}
.grain { width: 105px; height: 80px; right: 18px; bottom: 28px; border-radius: 50%; background: #e7c980; }
.greens { width: 98px; height: 100px; top: 30px; right: 20px; border-radius: 48% 52% 45% 55%; background: #62a66d; }
.protein { width: 102px; height: 74px; left: 34px; bottom: 28px; border-radius: 42% 58% 48% 52%; background: #d9a675; }
.leaf-one { width: 28px; height: 55px; top: 8px; left: 98px; z-index: 2; border-radius: 100% 0 100% 0; background: #326f4f; transform: rotate(30deg); }
.leaf-two { width: 22px; height: 46px; top: 11px; left: 122px; z-index: 2; border-radius: 0 100% 0 100%; background: #4d9264; transform: rotate(-18deg); }
.mini-card {
  position: absolute; padding: 12px 16px; color: var(--muted); border: 1px solid var(--border);
  border-radius: 14px; background: rgba(255, 255, 255, .94); font-size: .78rem;
  line-height: 1.35; box-shadow: 0 12px 30px rgba(30, 70, 55, .12);
}
.mini-card strong { color: var(--green-900); font-size: .9rem; }
.mini-protein { top: 20px; right: 0; }
.mini-fresh { bottom: 8px; left: 0; }

.workspace {
  width: min(1160px, calc(100% - 40px)); margin: 0 auto 70px; display: grid;
  grid-template-columns: minmax(310px, .78fr) minmax(0, 1.22fr); gap: 26px; align-items: start;
}
.form-panel, .results-panel { border: 1px solid var(--border); border-radius: 22px; background: var(--white); box-shadow: var(--shadow); }
.form-panel { padding: 30px; }
.results-panel { min-height: 500px; padding: 30px; }
h2 { margin-bottom: 8px; color: var(--green-900); font: 1.7rem/1.25 Georgia, "Times New Roman", serif; }
.panel-intro { margin-bottom: 24px; color: var(--muted); font-size: .92rem; }
```

```
.field { margin-bottom: 18px; }
.input-row { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
label { display: block; margin-bottom: 7px; font-size: .88rem; font-weight: 700; }
input, select, button { width: 100%; min-height: 48px; border-radius: 10px; font: inherit; }
input, select { padding: 10px 12px; color: var(--charcoal); border: 1px solid #cfdad3; background: white; }
.input-with-unit { position: relative; }
.input-with-unit input { padding-right: 48px; }
.input-with-unit span { position: absolute; top: 50%; right: 12px; color: var(--muted); font-size: .78rem; transform: translateY(-50%); }
input:focus, select:focus, button:focus-visible, a:focus-visible { outline: 3px solid rgba(69, 165, 121, .35); outline-offset: 2px; }
.form-error { min-height: 24px; margin-bottom: 9px; color: var(--error); font-size: .84rem; font-weight: 650; }
button { padding: 12px 18px; color: white; border: 0; background: var(--green-700); font-weight: 750; cursor: pointer; transition: background 160ms ease;
    }
button:disabled { cursor: wait; opacity: .7; }

.results-heading { display: flex; justify-content: space-between; gap: 16px; margin-bottom: 22px; }
.result-count { height: fit-content; padding: 6px 10px; color: var(--green-700); border-radius: 999px; background: var(--green-100); font-size: .78rem;
    font-weight: 800; white-space: nowrap; }
.results { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 16px; }
.state-card { min-height: 330px; grid-column: 1 / -1; padding: 38px; display: grid; place-content: center; text-align: center; border: 1px dashed #c9d8ce;
    border-radius: 17px; background: #fafcf9; }
.state-icon { width: 54px; height: 54px; margin: 0 auto 18px; display: grid; place-items: center; color: var(--green-700); border-radius: 50%; background:
    var(--green-100); font-size: 1.35rem; font-weight: 900; }
.state-card h3 { margin-bottom: 7px; color: var(--green-900); font-size: 1rem; }
.state-card p { margin-bottom: 0; color: var(--muted); font-size: .88rem; }
.food-card { padding: 20px; border: 1px solid var(--border); border-radius: 17px; background: white; }
.food-topline { display: flex; align-items: start; justify-content: space-between; gap: 12px; margin-bottom: 9px; }
.food-card h3 { margin-bottom: 0; color: var(--green-900); font-size: 1.05rem; line-height: 1.35; }
.category { padding: 4px 8px; color: var(--green-700); border-radius: 999px; background: var(--green-100); font-size: .7rem; font-weight: 800;
    white-space: nowrap; }
.description { min-height: 44px; margin-bottom: 16px; color: var(--muted); font-size: .82rem; }
.nutrition-grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 8px; }
.nutrition-item { padding: 9px; border-radius: 9px; background: var(--sage-50); }
.nutrition-item span, .nutrition-item strong { display: block; }
.nutrition-item span { color: var(--muted); font-size: .67rem; }
.nutrition-item strong { color: var(--green-900); font-size: .88rem; }

footer { padding: 28px 20px 38px; text-align: center; color: var(--muted); border-top: 1px solid var(--border); font-size: .8rem; }
footer p { margin-bottom: 4px; }

@media (max-width: 850px) {
  .hero { min-height: auto; padding: 50px 42px; grid-template-columns: 1fr; }
  .hero-visual { min-height: 300px; }
  .workspace { grid-template-columns: 1fr; }
  .results-panel { min-height: 440px; }
}

@media (max-width: 560px) {
  .site-header, .hero, .workspace { width: min(100% - 28px, 1160px); }
  .site-header { padding: 18px 0; }
  .header-link { display: none; }
  .hero { margin-bottom: 38px; padding: 38px 24px; border-radius: 24px; }
  h1 { font-size: 2.55rem; }
  .hero-visual { min-height: 235px; }
  .plate { width: 210px; height: 210px; border-width: 17px; }
  .food-shape, .leaf { transform: scale(.72); transform-origin: top left; }
  .tomato { top: 12px; left: 12px; }
  .greens { top: 18px; right: -4px; }
  .grain { right: -4px; bottom: -2px; }
  .protein { left: 20px; bottom: 0; }
  .leaf-one { top: 4px; left: 72px; transform: rotate(30deg) scale(.72); }
  .leaf-two { top: 7px; left: 90px; transform: rotate(-18deg) scale(.72); }
  .mini-card { padding: 9px 11px; font-size: .68rem; }
  .mini-protein { right: -4px; }
  .mini-fresh { left: -4px; }
  .form-panel, .results-panel { padding: 22px; border-radius: 18px; }
  .input-row, .results { grid-template-columns: 1fr; }
  .state-card { min-height: 270px; padding: 26px 18px; }
  .description { min-height: auto; }
}

@media (prefers-reduced-motion: reduce) {
  html { scroll-behavior: auto; }
  * { transition: none !important; }
}
```

5. public/app.js

```
const form = document.querySelector("#nutrition-form");
const goalInput = document.querySelector("#goal");
const calorieInput = document.querySelector("#max-calories");
const proteinInput = document.querySelector("#min-protein");
const submitButton = document.querySelector("#submit-button");
const formError = document.querySelector("#form-error");
const results = document.querySelector("#results");
const resultCount = document.querySelector("#result-count");

function showFormError(message) {
  formError.textContent = message;
}

function validateInputs(goal, maxCalories, minProtein) {
  if (!goal) {
    return "Please select a nutrition goal.";
  }

  if (maxCalories === "" || Number(maxCalories) <= 0) {
    return "Please enter a valid calorie limit greater than zero.";
  }

  if (minProtein === "" || Number(minProtein) < 0) {
    return "Please enter a valid minimum protein value.";
  }

  return "";
}

function setLoading(isLoading) {
  submitButton.disabled = isLoading;
  submitButton.textContent = isLoading ? "Finding Foods..." : "Recommend Foods";
  results.setAttribute("aria-busy", String(isLoading));
}

function clearResults() {
  results.replaceChildren();
  resultCount.hidden = true;
}

function createStateCard(title, message, symbol) {
  const card = document.createElement("div");
  card.className = "state-card";

  const icon = document.createElement("span");
  icon.className = "state-icon";
  icon.setAttribute("aria-hidden", "true");
  icon.textContent = symbol;

  const heading = document.createElement("h3");
  heading.textContent = title;

  const text = document.createElement("p");
  text.textContent = message;

  card.append(icon, heading, text);
  return card;
}

function createNutritionItem(label, value) {
  const item = document.createElement("div");
  item.className = "nutrition-item";

  const labelElement = document.createElement("span");
  labelElement.textContent = label;

  const valueElement = document.createElement("strong");
  valueElement.textContent = value;

  item.append(labelElement, valueElement);
  return item;
}

function createFoodCard(food) {
```

```
  const card = document.createElement("article");
  card.className = "food-card";

  const topline = document.createElement("div");
  topline.className = "food-topline";

  const name = document.createElement("h3");
  name.textContent = food.name;

  const category = document.createElement("span");
  category.className = "category";
  category.textContent = food.category;
  topline.append(name, category);

  const description = document.createElement("p");
  description.className = "description";
  description.textContent = food.description;

  const nutrition = document.createElement("div");
  nutrition.className = "nutrition-grid";
  nutrition.append(
    createNutritionItem("Calories", `${food.calories} kcal`),
    createNutritionItem("Protein", `${food.protein} g`),
    createNutritionItem("Carbs", `${food.carbs} g`),
    createNutritionItem("Fat", `${food.fat} g`)
  );

  card.append(topline, description, nutrition);
  return card;
}

function renderFoods(foods) {
  clearResults();
  resultCount.textContent = `${foods.length} ${foods.length === 1 ? "match" : "matches"}`;
  resultCount.hidden = false;

  foods.forEach((food) => {
    results.append(createFoodCard(food));
  });
}

function renderNoResults() {
  clearResults();
  results.append(
    createStateCard(
      "No exact food match found.",
      "Try increasing the calorie limit or lowering the minimum protein requirement.",
      "!"
    )
  );
}

function renderRequestError() {
  clearResults();
  results.append(
    createStateCard(
      "Unable to load food recommendations right now.",
      "Please try again.",
      "!"
    )
  );
}

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const goal = goalInput.value;
  const maxCalories = calorieInput.value;
  const minProtein = proteinInput.value;
  const validationMessage = validateInputs(goal, maxCalories, minProtein);

  showFormError(validationMessage);
  if (validationMessage) {
    return;
  }
```

```
  setLoading(true);

  try {
    const response = await fetch("/api/recommend", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        goal,
        maxCalories: Number(maxCalories),
        minProtein: Number(minProtein)
      })
    });

    const data = await response.json();

    if (!response.ok) {
      showFormError(data.message || "Please check the entered values.");
      return;
    }

    if (data.count === 0) {
      renderNoResults();
    } else {
      renderFoods(data.foods);
    }
  } catch (error) {
    renderRequestError();
  } finally {
    setLoading(false);
  }
});
```